

## บทที่ 2

### หลักการและทฤษฎี

#### 2.1 โปรแกรมที่ใช้ในการทำโครงงาน

##### 2.1.1 โครงสร้างพื้นฐานของเวิลด์ไวด์เว็บ (World Wide Web , www)

เวิลด์ไวด์เว็บ (WWW) นั้นแปลความหมายเป็นภาษาไทยอย่างตรงๆ แปลความได้ว่า สายใยกว้างไกล ครอบคลุม ทั่วโลก หรือเครือข่ายใยแมงมุม

เวิลด์ไวด์เว็บ (WWW) เป็นการบริการข้อมูลข่าวสารที่มีเขตครอบคลุมทั่วทั้งโลก ในแบบสื่อผสมหรือมัลติมีเดีย (Multimedia) กล่าวคือ ข้อมูลเหล่านี้จะประกอบไปด้วยข้อความ ภาพ และเสียงประกอบกัน จึงเป็นที่นิยมของคนทั้งโลก ข้อมูลจะถูกแบ่งออกเป็นหน้าๆ โดยแต่ละหน้าจะถูกเขียนขึ้นโดยภาษาทางคอมพิวเตอร์ และข้อมูลต่างๆ สามารถเชื่อมโยงถึงกันได้โดยไม่จำเป็นต้องอยู่ที่เดียวกัน

เว็บ คือ รูปแบบของข้อมูลที่ใช้ในการติดต่อสื่อสารบนเครือข่ายอินเทอร์เน็ต ซึ่งเกี่ยวข้องกับหลายๆ ส่วน มีส่วนที่สำคัญ ดังนี้

เวิลด์ไวด์เว็บเซิร์ฟเวอร์ (WWW Server) เป็นแหล่งข้อมูลในระบบ หมายถึง เครื่องคอมพิวเตอร์หรือศูนย์คอมพิวเตอร์ในเครือข่ายอินเทอร์เน็ต ซึ่งเซิร์ฟเวอร์จะให้บริการข้อมูลที่เรียกว่า ข้อมูลเอชทีเอ็มแอล (HTML)

เอชทีเอ็มแอล (HTML) เป็นภาษาสำหรับเขียนข้อมูลแบบไฮเปอร์เท็กซ์ ซึ่งเป็นไฟล์ข้อมูลที่ใช้ในระบบเวิลด์ไวด์เว็บ หรือไฟล์แสดงโฮมเพจ ดังนั้นจึงเรียกไฟล์ข้อมูลชนิดนี้ว่า ไฟล์ข้อมูลเอชทีเอ็มแอล ซึ่งไฟล์ชนิดนี้ประกอบด้วยข้อมูลประเภทต่างๆ ได้แก่ ข้อความ รูปภาพ เสียง และวิดีโอ เป็นต้น ไฟล์เอชทีเอ็มแอลที่แสดงผ่านโปรแกรมเบราว์เซอร์เช่นนี้ถูกเรียกว่า โฮมเพจ

ไฮเปอร์ลิงก์ (Hyperlink) เป็นคำหรือวลีของเว็บเพจ หรือ โฮมเพจ ซึ่งเกิดจากไฟล์ข้อมูล เอชทีเอ็มแอล ส่วนไฮเปอร์เท็กซ์นั้นเป็นคำหรือวลีพิเศษที่เป็นจุดเชื่อมโยงแหล่งข้อมูลได้โดยใช้เมาส์คลิกไปยังคำหรือวลีนั้นๆ

เว็บไซต์ (Web Site) เป็นเครื่องมือที่ใช้จัดเก็บเว็บเพจของแต่ละองค์กร ที่จะนำเสนอข้อมูลของตนในรูปแบบของเว็บ องค์กรต่างๆ มักจะมีเว็บไซต์เป็นของตนเอง และมักใช้ชื่อองค์กรเป็นชื่อเว็บไซต์ เพื่อให้สามารถจดจำได้ง่าย นอกจากนี้ยังจัดมีกลุ่มคนที่มีความสนใจในเรื่องคล้ายๆ กัน หรือกลุ่มคนที่ต้องการเผยแพร่ความรู้ เทคนิคต่างๆ จึงได้ตั้งเว็บไซต์ขึ้นมาด้วย

โฮมเพจ (Home Page) คือเว็บเพจหน้าแรกของแต่ละเรื่อง ซึ่งเปรียบเสมือนหน้าปกของหนังสือแต่ละเล่ม ซึ่งในส่วนของโฮมเพจนี้จะเป็นที่ส่วนที่บอกให้ทราบว่าข้อมูลในเว็บไซต์นั้นๆ เป็นข้อมูลที่เกี่ยวกับเรื่องใด พร้อมกับมีสารบัญในการเลือกไปยังหัวข้อต่างๆ

เว็บเพจ (Web Page) คือเอกสารข้อมูลในแต่ละหน้า ซึ่งจะประกอบไปด้วย รูปภาพ ภาพเคลื่อนไหว และเสียง จึงเป็นข้อมูลแบบผสม หรือมัลติมีเดียนั่นเอง

เว็บเบราว์เซอร์ (Web Browser) เป็นโปรแกรมที่ทำหน้าที่อ่านและแสดงผลเว็บเพจซึ่งเป็นเอกสารข้อมูลที่เขียนขึ้นโดยภาษาเอชทีเอ็มแอล เช่น โปรแกรม อินเทอร์เน็ตเอกซ์พลอเรอร์ (Internet Explorer), Netscape, นีโอพลาเน็ต (NeoPlanet), โอเปรา (Opera) และไทยเบราว์เซอร์ (ThaiBrowser) เป็นต้น

โปรโตคอลเอชทีทีพี (HTTP-Protocol HTTP) นี้สร้างขึ้นสำหรับการบริการเว็บไซต์เว็บในเครือข่ายอินเทอร์เน็ต โดยเฉพาะโปรโตคอลตัวนี้จะเป็นตัวกำหนดวิธีในการส่งข้อมูลหรือไฟล์ระหว่างเครื่องคอมพิวเตอร์ที่เป็นเซิร์ฟเวอร์ รวมถึงกฎระเบียบในการติดต่อด้วย

### 2.1.2 หลักการทำงานของโปรโตคอลเอชทีทีพี (HTTP – Hypertext Transfer Protocol)

เครือข่ายเว็บไซต์เว็บ (World Wide Web) จะเชื่อมโยงกันโดยใช้ HTTP โปรโตคอล ซึ่ง HTTP โปรโตคอลนี้เป็นโปรโตคอลที่เกี่ยวกับการจัดการเครือข่ายที่ใช้ในการติดต่อสื่อสารและรับส่งข้อมูลภายใต้ระบบเว็บซึ่งก็คือ ไฮเปอร์เท็ก หรือ เว็บเพจนั่นเอง โดยถูกออกแบบให้มีความกะทัดรัด สามารถทำงานได้รวดเร็ว มีกระบวนการทำงานที่ไม่ซับซ้อนมากนัก แต่สามารถรองรับข้อมูลได้ทุกแบบ ไม่ว่าจะเป็นข้อมูลทั่วไปที่เข้ารหัสแบบ เอ็มไอเอ็มอี (MIME) หรือข้อมูลที่เป็นกราฟฟิก เช่น ไฟล์ที่เป็นจีไอเอฟ (GIF) หรือ เจเปก (JPEG) เป็นต้น ซึ่งข้อมูลต่างๆที่เป็นส่วนประกอบของโฮมเพจที่ร้องขอบริการ จะมีการเปิดการติดต่อใหม่เป็นอิสระแก่กัน

โดยรูปแบบจะเป็นแบบ Connection Oriented และการทำงานพื้นฐานจะมีรูปแบบเป็นลักษณะแบบ Transaction Oriented คือจะอาศัยหลักการง่ายๆของไคลเอนต์-เซิร์ฟเวอร์ (Client/Server) ในการร้องขอบริการ โดยหลักการการทำงานจะแบ่งออกเป็น 2 ด้าน คือ ด้านเว็บเซิร์ฟเวอร์และด้านไคลเอนต์ โดยไคลเอนต์จะติดต่อเข้ามายังเซิร์ฟเวอร์โดยใช้โปรแกรมเบราว์เซอร์และอ้างถึงแอดเดรสของเซิร์ฟเวอร์โดยใช้รูปแบบของยูอาแอล (URL) ส่วนด้านเซิร์ฟเวอร์จะส่งข้อมูลกลับมาในรูปแบบที่เป็นภาษาเอชทีเอ็มแอล (HTML – Hypertext Markup Language) โดยที่โปรโตคอลเอชทีทีพีใช้วิธีการเข้ารหัสในแบบ เอ็มไอเอ็มอี (MIME) เป็นมาตรฐานในการทำงาน

#### 2.1.2.1 วิธีการติดต่อของโปรโตคอลเอชทีทีพี (HTTP)

การทำงานของโปรโตคอลเอชทีทีพี (HTTP) นั้นเป็นแบบไคลเอนต์และเซิร์ฟเวอร์ ดังนั้นการสื่อสารใดๆ ผ่านโปรโตคอลนี้จำเป็นต้องมีเครื่องคอมพิวเตอร์ตัวลูกกับตัวแม่ การสื่อสารจึงจะสมบูรณ์ได้ในการติดต่อกันระหว่างไคลเอนต์ไปยังเซิร์ฟเวอร์ผ่านโปรโตคอลเอชทีทีพีอย่างเช่นการเปิดเว็บเพจผ่านเว็บเบราว์เซอร์ จะมีขั้นตอน ดังนี้



รูปที่ 2-1 การติดต่อระหว่างไคลเอนต์กับเซิร์ฟเวอร์

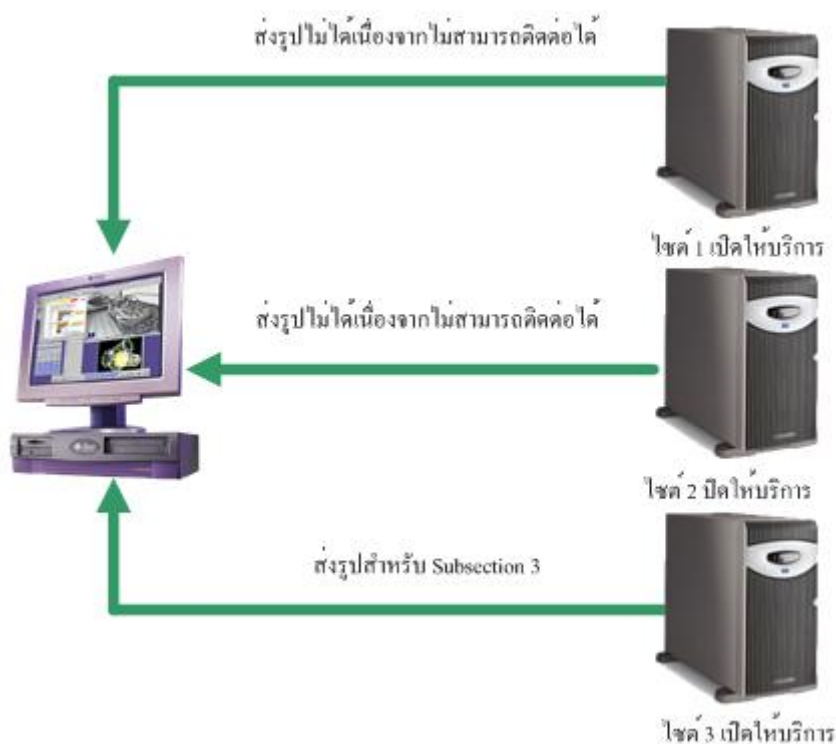
จากรูปที่ 2-1 ขั้นแรกคือ ไคลเอนต์ (ในตอนนี้คือเว็บเบราว์เซอร์) จะสร้างการเชื่อมต่อ (Connection) กับเซิร์ฟเวอร์ผ่านสิ่งที่เรียกว่า “ซ็อกเก็ต” (socket) เมื่อซ็อกเก็ตทั้งสองฝั่งเชื่อมต่อกันได้สำเร็จ ไคลเอนต์จะส่งคำร้องขอข้อมูล (request) ไปยังเซิร์ฟเวอร์ จากนั้นเซิร์ฟเวอร์จะไปหาข้อมูลที่ไคลเอนต์ต้องการ ไม่ว่าจะมีความรู้หรือไม่มีก็ตามดังที่ไคลเอนต์ร้องขอมา เซิร์ฟเวอร์ก็จะส่งข้อมูลตอบสนอง (response) กลับมายังไคลเอนต์เสมอ สุดท้ายการเชื่อมต่อจะถูกตัดขาดหรือปลดการเชื่อมต่อของซ็อกเก็ตทั้งสองฝั่งออก

การทำงานของโปรโตคอลเอชทีทีพี ที่มีการเชื่อมต่อในระยะเวลาเพียงสั้นๆ หรือที่เรียกว่า โปรโตคอลแบบ connectionless ทำให้ช่วงเวลานั้นๆ เซิร์ฟเวอร์ที่ให้บริการนั้นสามารถรองรับไคลเอนต์ได้จำนวนมากพร้อมๆ กัน เพราะไม่มีไคลเอนต์ใดได้ทำการเชื่อมต่ออย่างถาวร

#### 2.1.2.2 การจัดการข้อมูลของโปรโตคอลเอชทีทีพี (HTTP Protocol)

โปรโตคอลเอชทีทีพี ช่วยจัดการเรื่องของข้อมูลที่ใช้ในการรับและส่งไฟล์ เพราะโปรโตคอลนี้จะระบุประเภทข้อมูล (data type) มากับข้อมูลเสมอ ทำให้หมดปัญหาเรื่องการตีความหมายของเนื้อไฟล์ไป ดังนั้นเราจึงสามารถส่งไฟล์ทุกประเภทผ่านทางโปรโตคอลนี้ ถ้าฝ่ายไคลเอนต์ได้รับไฟล์ไปแล้วทราบถึงวิธีการแสดงผลของไฟล์นั้น ก็จัดการตามกรรมวิธีของไฟล์นั้นได้เลย แต่ถ้าหากไม่ทราบแล้วนั้นก็จะเป็นหน้าที่ของผู้ใช้เองที่ต้องหาโปรแกรม หรือแอปพลิเคชันมาจัดการกับไฟล์นั่นเอง

ตัวอย่างเช่น การเปิดไฟล์ผ่านเว็บเบราว์เซอร์ หากเป็นไฟล์นามสกุลคอตเอชทีเอ็ม (.htm) , คอตเอชทีเอ็มแอล (.html) , คอตเอชทีเอ็ม (.htm) หรือคอตทีเอกที (.txt) แล้วเว็บเบราว์เซอร์จะนำข้อมูลออกแสดงทางวินโดว์ได้ทันที แต่ถ้าเป็นเป็นไฟล์นามสกุลพีดีเอฟ (.pdf) เว็บเบราว์เซอร์จะความีโปรแกรมเสริมจำพวก ปลั๊กอิน (plug-in) สำหรับเปิดดูไฟล์ชนิดนี้หรือไม่ ถ้ามีก็จะทำการเรียกปลั๊กอินให้มาช่วยนำข้อมูลไปแสดงผล ถ้าไม่มีปลั๊กอินหรือไม่เคยรู้จักนามสกุลนั้นมาก่อน ก็จะแสดงไดอะล็อกบ็อกซ์ ถามความต้องการผู้ใช้อีกทีหนึ่งว่าจะจัดการต่อไปอย่างไร

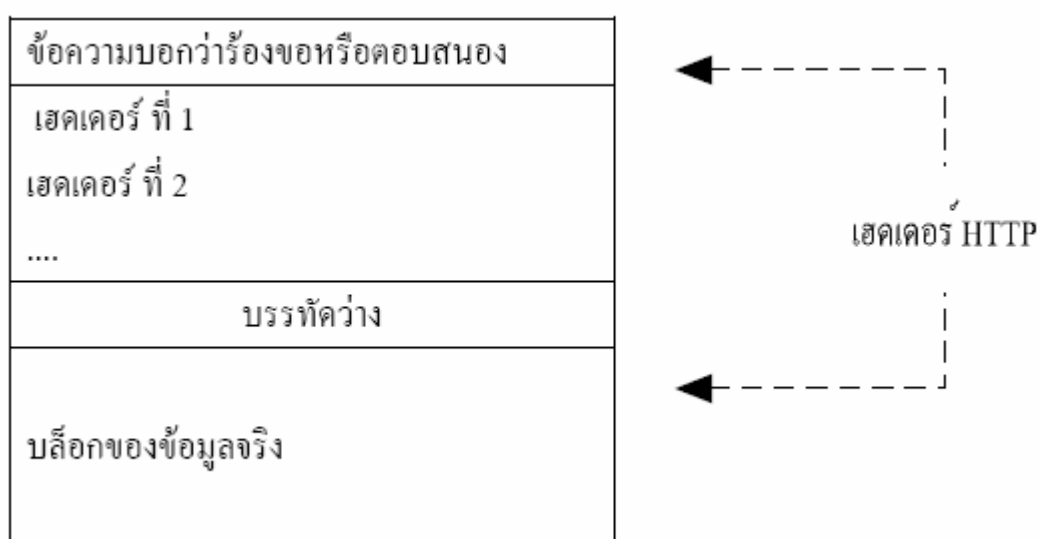


รูปที่ 2-2 การเปิดการติดต่อในการร้องขอโฮมเพจ

จากรูปเป็นการแสดงถึงโฮมเพจที่มีการติดต่อขอข้อมูลจากไชต์อื่นๆ 3 ไชต์ โดยที่ไคลเอนต์ (หรือบราวเซอร์) ทำการร้องขอข้อมูลจากไชต์ ที่แตกต่างกัน 3 ไชต์ ซึ่งตัวข้อมูลที่ไคลเอนต์ร้องขอข้อมูลในแต่ละไชต์และแต่ละงานงานนั้นจะมีอิสระแก่กัน นั่นคือ การร้องขอจากไชต์ 1 ไคลเอนต์เปิดการติดต่อกับไชต์ที่ 1 โดยติดต่อผ่านเว็บเซิร์ฟเวอร์เพื่อทำการร้องขอข้อมูลที่ต้องการ ซึ่งเซิร์ฟเวอร์ HTTPD คอยให้บริการที่พอร์ต 80 แล้วเซิร์ฟเวอร์ ก็จะส่งข้อมูลกลับตามที่ไฟล์ขอมาหลังจากนั้นจึงปิดการติดต่อ การร้องขอบริการจากไชต์ 2 ไคลเอนต์ทำการขอไปยังไชต์ 2 แต่ว่า ไชต์ 2 ปิดการให้บริการอยู่ จึงไม่สามารถติดต่อได้ หลังจากนั้นจึงปิดการติดต่อ การร้องขอบริการจากไชต์ 3 ไคลเอนต์ทำการร้องขอไปยังไชต์ 3 ซึ่งมีการทำงานเหมือนกับไชต์ 1 แต่แตกต่างกันตรงตัวข้อมูลที่ส่งกลับ ไชต์ 3 สามารถให้บริการได้เซิร์ฟเวอร์ก็จะส่งข้อมูลกลับมายังไคลเอนต์ตามที่ไคลเอนต์ได้ร้องขอ จากการร้องขอข้อมูลของไฟล์ไปยังไชต์ 1, 2 และ 3 จะเห็นว่าข้อมูลที่ส่งกลับมาจากแต่ละไชต์ไม่ขึ้นแก่กันทั้ง 3 ไชต์ ไชต์ 1 และ 3 เว็บเซิร์ฟเวอร์มีการส่งข้อมูลกลับมายังไคลเอนต์ได้ แต่ไชต์ 2 ไม่มีข้อมูลกลับมา ซึ่งหลักการที่สำคัญในการทำงานเป็นเรื่องของการร้องขอของไคลเอนต์ และการตอบกลับของเซิร์ฟเวอร์นั้นใช้หลักการของ ไคลเอนต์เซิร์ฟเวอร์ ทั้งนี้เพราะ HTTP ก็เป็นโพรโทคอล ที่ทำงานแบบไฟล์เซิร์ฟเวอร์ และด้วยเทคนิคกับวิธีที่กล่าวไว้ข้างต้น จะเห็นว่าจะมีข้อดีคือ ทำให้งานแต่ละงานเป็นอิสระต่อกันดังนั้นหากมีส่วนใดเสียหรือมีปัญหาในการติดต่อสื่อสารไม่ว่าจะด้วยสาเหตุใดก็ตามจะไม่กระทบแก่กัน

ในการร้องขอจากไคลเอนต์ และตอบสนองจากเซิร์ฟเวอร์ ย่อมต้องมีการรับส่งข้อมูลระหว่างกัน แต่ข้อมูลที่รับส่งให้กันในแต่ละครั้งนั้นไม่ได้มีเฉพาะข้อมูลเพียงอย่างเดียว แต่ละฝ่ายจะต้องเพิ่มส่วนที่เรียกว่าเฮดเดอร์เอชทีทีพี (HTTP Header) เข้าไปในส่วนหัวของข้อมูลด้วย เฮดเดอร์เอชทีทีพี จะใช้เป็นตัวบอกว่าข้อมูลที่ส่งเป็นข้อมูลการร้องขอจากไคลเอนต์หรือเป็นการตอบสนองจากเซิร์ฟเวอร์นั่นเอง

เนื่องจากข้อมูลในเฮดเดอร์เอชทีทีพี เป็นตัวควบคุมหรือบอกว่าให้ฝ่ายรับควรทำอะไรกับข้อมูลที่ส่งมาให้ ในบางครั้งจึงมีคนเรียกข้อมูลในส่วนหัวนี้ว่าข้อมูลเมตา (Meta Information) ถึงแม้ว่าข้อมูลในเฮดเดอร์เอชทีทีพีจะสำคัญต่อการนำไปตีความหมายของข้อมูลแล้วก็ตาม แต่ข้อมูลในเฮดเดอร์เอชทีทีพี จะเป็นเพียงข้อความธรรมดา มิได้มีการเข้ารหัสหรือมีลักษณะพิเศษแตกต่างจากข้อมูลปกติเลยแต่อย่างใด โดยโครงสร้างจะเป็นดังรูปที่ 2-3



รูปที่ 2-3 โครงสร้างของข้อมูลที่ส่งผ่านโปรโตคอลเอชทีทีพี

#### 2.1.2.3 ข้อความร้องขอ (request)

เมื่อไคลเอนต์เชื่อมต่อกับเซิร์ฟเวอร์สำเร็จแล้ว ไคลเอนต์ต้องเป็นฝ่ายเริ่มเปิดการพูดคุยก่อน ด้วยการส่งข้อมูลไปยังเซิร์ฟเวอร์เพื่อบอกการร้องขอข้อมูล การร้องขอข้อมูลไปยังเซิร์ฟเวอร์สามารถทำได้หลายแบบ เช่น ร้องขอให้เซิร์ฟเวอร์ส่งไฟล์มาให้แบบนี้จะเรียกว่าร้องขอแบบ GET , หรือร้องขอเพื่อถามว่าเซิร์ฟเวอร์ว่ามีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์หรือไม่ (ถามเฉยๆ เท่านั้นไม่ต้องการให้เซิร์ฟเวอร์ส่งไฟล์จริงมาให้) แบบนี้จะเรียกว่าร้องขอแบบ HEAD , หากเป็นการร้องขอให้เซิร์ฟเวอร์รับข้อมูลจากไคลเอนต์เรียกว่าร้องขอแบบ POST , ซึ่งการร้องขอแบบนี้หมายความว่าไคลเอนต์ต้องการส่งข้อมูลไปให้เซิร์ฟเวอร์นั่นเอง แต่ในความเป็นจริงเราสามารถส่งข้อมูลไปยังเซิร์ฟเวอร์ด้วยการร้องขอแบบ GET ก็ได้

วิธีการร้องขอมีอยู่หลายวิธีขึ้นอยู่กับเวอร์ชันของโปรโตคอลเอชทีทีพี ที่ใช้ หากเป็นเวอร์ชัน 1.0 จะมีวิธีร้องขอมาตรฐาน 3 วิธี คือ GET , HEAD และ POST แต่ถ้าใช้เวอร์ชัน 1.1 จะมีวิธีการร้องขอเพิ่มจากเวอร์ชัน 1.0 อีกหลายวิธี เช่น OPTION , PUT , DELETE , TRACE เป็นต้น

ซึ่งการใช้โปรโตคอลเอชทีทีพีเวอร์ชันไหนนั้น ขึ้นอยู่กับเซิร์ฟเวอร์และไคลเอนต์ที่จะทำงานด้วย เช่น ถ้าทราบว่าเซิร์ฟเวอร์ที่จะติดต่อด้วยนั้น ใช้เว็บเซิร์ฟเวอร์ที่ทำงานสนับสนุนเอชทีทีพี 1.1 ดังนั้นวิธีการร้องขอก็สามารถใช้ของเวอร์ชัน 1.1 ได้

ในส่วนของฝั่งไคลเอนต์ โปรแกรมเว็บเบราว์เซอร์รุ่นใหม่ ๆ จะสนับสนุนเอชทีทีพีเวอร์ชัน 1.1 อยู่แล้ว ส่วนในกรณีที่เขียนซีจีไอ (CGI) เพื่อทำงานสร้างคำร้องไปยังเซิร์ฟเวอร์นั้นก็ไม่มีปัญหาใดๆ โดยสามารถเขียนให้ซีจีไอสร้างคำร้องขอแบบใดก็ได้ เพราะเป็นข้อความบรรทัดเดียว ดังนั้นปัญหาจึงขึ้นอยู่กับเซิร์ฟเวอร์ที่จะติดต่อด้วยมากกว่า ว่าเซิร์ฟเวอร์จะรู้จักและสนับสนุนเอชทีทีพีเวอร์ชัน 1.1 หรือเวอร์ชันที่ใหม่กว่าได้หรือไม่

เพื่อไม่ให้เกิดปัญหาในการทำงาน ดังนั้นเราจึงควรใช้โปรโตคอลเอชทีทีพีที่เป็นเวอร์ชันกลางๆ อย่างเช่น เอชทีทีพีเวอร์ชัน 1.0 ซึ่งเซิร์ฟเวอร์ส่วนใหญ่จะใช้เวอร์ชันนี้กัน ในทางปฏิบัตินั้นเฉพาะวิธีการร้องขอ 3 วิธีของเวอร์ชัน 1.0 ผสมกับการเขียนซีจีไอ ก็สามารถทำงานได้อย่างเพียงพอแล้ว

#### 2.1.2.3.1 การร้องขอด้วยเมธอดเก็ท (GET)

ตัวอย่างในกรณีที่เว็บเบราว์เซอร์ขอเปิดเอกสารเอชทีเอ็มแอล จากเซิร์ฟเวอร์จะร้องขอแบบเก็ท (GET) โดยเมื่อเว็บเบราว์เซอร์สร้างซ็อกเก็ตเชื่อมต่อกับเซิร์ฟเวอร์ได้แล้ว จะสร้างข้อความร้องขอขึ้นมาเพื่อส่งไปยังเซิร์ฟเวอร์ ข้อความในบรรทัดแรกของเฮดเดอร์เอชทีทีพี (HTTP Header) จะประกอบไปด้วยส่วน 3 ส่วน คือ วิธีการร้องขอ หรือที่เรียกว่า “เมธอด” (Method) , ไคลเอนต์กับชื่อไฟล์ที่ร้องขอ และเวอร์ชันของเอชทีทีพี ที่ผู้ร้องขอใช้อยู่แต่ละส่วนของข้อความจะถูกคั่นด้วยช่องว่าง (Space) ฉะนั้นรูปแบบในการเขียนข้อความบรรทัดแรกของเฮดเดอร์เอชทีทีพี จึงเป็นดังนี้

---

GET /path/file HTTP / x.x

---

|            |                                                                                       |
|------------|---------------------------------------------------------------------------------------|
| Get        | บอกว่าใช้เมธอดเก็ทในการร้องขอข้อมูลจากเซิร์ฟเวอร์                                     |
| /path/file | ไคลเอนต์และชื่อไฟล์ที่ต้องการจากเซิร์ฟเวอร์                                           |
| HTTP / x.x | เวอร์ชันของเอชทีทีพี บอกแก่เซิร์ฟเวอร์ให้รู้ว่าทางไคลเอนต์ใช้เอชทีทีพีเวอร์ชันไหนอยู่ |

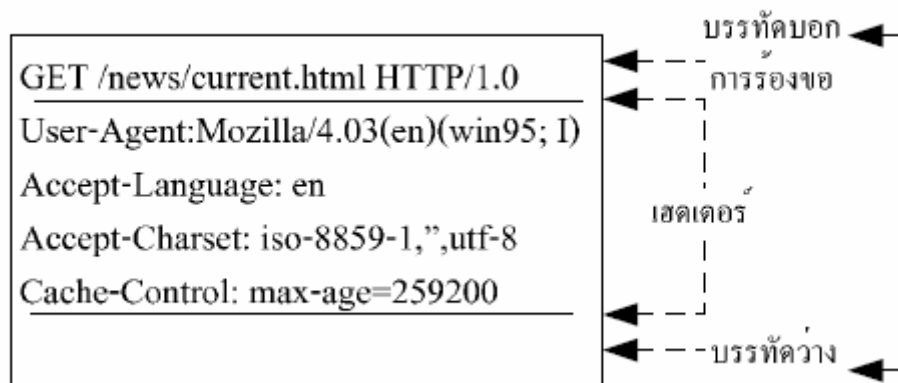
สังเกตได้ว่าการร้องขอข้อมูลจากเซิร์ฟเวอร์ จะระบุเฉพาะไคลเอนต์และชื่อไฟล์ที่ต้องการเท่านั้น ไม่มีการบอกชื่อโฮสต์และโดเมนเนมเพราะว่าเว็บเบราว์เซอร์ได้สร้างการเชื่อมต่อกับโฮสต์เอาไว้แล้ว การร้องขอข้อมูลจึงไม่ต้องระบุชื่อโฮสต์ซ้ำอีกครั้ง เราเรียกส่วนที่ระบุ /path/file นี้ว่า ยูอาร์ไอ (URI -

Uniform Resource Identifier) ซึ่งจะคล้ายกับ ยูอาร์แอล (URI - Uniform Resource Locator) แต่สั้นกว่า เนื่องจากว่าไม่ต้องบอกโปรโตคอล, ชื่อโฮสต์ และโดเมนเนมอย่างยาวเหยียดแต่อย่างใด

ตัวอย่างเช่น ถ้าเราป้อนยูอาร์แอลให้กับเว็บเบราว์เซอร์ เป็น

<http://www.ventura.lanna.com/news/current.html> เว็บเบราว์เซอร์จะต้องสร้างการเชื่อมต่อไปยัง

ventura.com ก่อนแล้วจึงร้องขอส่งไปยังเซิร์ฟเวอร์ โดยสมมุติว่าเว็บเบราว์เซอร์ที่ใช้นั้นเป็นเวอร์ชัน 1.0 ดังนั้นโครงสร้างของข้อความร้องขอจะเป็นดังรูปที่ 2-3

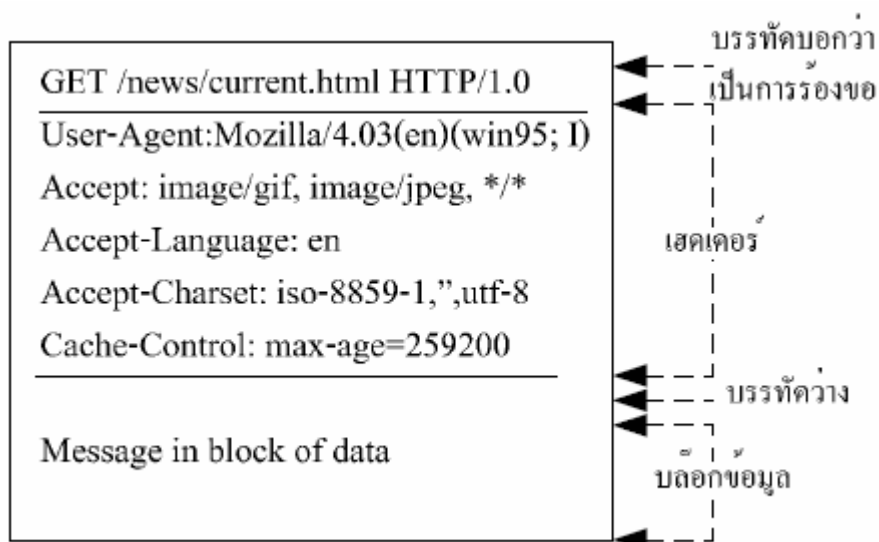


รูปที่ 2-4 ข้อความร้องขอที่ส่งไปยังเซิร์ฟเวอร์

โดยจากรูปที่ 2-3 นั้นตีความหมายจากเฮดเดอร์ไอซีทีพีได้ว่า เว็บเบราว์เซอร์ร้องขอเปิดไฟล์ current.html ในไดเรกทอรี /news (จากโฮสต์ ventura ที่มีโดเมนเนมเป็น lanna.com) และเว็บเบราว์เซอร์ใช้โปรโตคอลไอซีทีพีเวอร์ชัน 1.0 จะสังเกตว่า นอกจากคำร้องขอที่อยู่ในบรรทัดแรกแล้ว ยังมีข้อมูลอื่นๆ ที่เว็บเบราว์เซอร์ส่งไปให้เซิร์ฟเวอร์ด้วย ข้อมูลในส่วนที่สองนี้เรียกว่า “เฮดเดอร์” (Header) โดยข้อมูลในเฮดเดอร์ของเว็บเบราว์เซอร์แต่ละโปรแกรม และแต่ละเวอร์ชันจะแตกต่างกันออกไป

จากโครงสร้างของข้อมูลที่รับส่งกันระหว่างไคลเอนต์กับเซิร์ฟเวอร์ในรูปที่ 2-3 เราทราบว่า หลังจากเฮดเดอร์รายการสุดท้ายแล้ว จะมีบรรทัดว่างๆ แล้วตามด้วยบล็อกข้อมูล แต่เมื่อดูโครงสร้างของข้อความร้องขอที่ส่งไปยังเซิร์ฟเวอร์ในรูปที่ 2-4 แล้วนั้น หลังบรรทัดว่างๆ จะเห็นว่าไม่มีบล็อกข้อมูลเลย เนื่องมาจากในตอนนี้เป็นการร้องขอไฟล์ไปยังเซิร์ฟเวอร์ จึงไม่มีความจำเป็นที่จะต้องส่งอะไรไปให้เซิร์ฟเวอร์มากกว่านี้เพราะสิ่งที่ต้องการจากเซิร์ฟเวอร์ได้ระบุไว้ในคำร้องขอบรรทัดแรกสุดแล้ว

ตัวอย่างในรูปที่ 2-5 นั้นเป็นการส่งข้อความร้องขอไปยังเซิร์ฟเวอร์ด้วยเมธอด GET และมีการส่งข้อความว่า “Message in block of data” ไปให้เซิร์ฟเวอร์ด้วย แต่ถึงแม้จะทำแบบนี้ ก็ไม่เกิดประโยชน์เลย เนื่องจากทางเซิร์ฟเวอร์ไม่ดึงเอาข้อความนั้นไปใช้งานเลยแต่อย่างใด



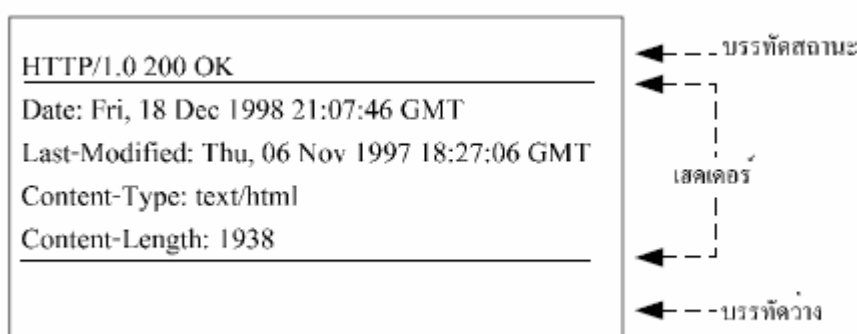
รูปที่ 2-5 ข้อความร้องขอที่มีข้อความส่งไปในบล็อกข้อมูลด้วย

#### 2.1.2.3.2 การร้องขอด้วยเมธอดเฮด (HEAD)

การร้องขอด้วยเมธอดเฮด (HEAD) จะคล้ายกับแบบเมธอดเก็ท (GET) คือใช้ร้องขอเพื่อถามเซิร์ฟเวอร์ และให้เซิร์ฟเวอร์ส่งรายละเอียดของไฟล์ที่ไคลเอ็นต์ต้องการกลับมา เพียงแต่เมธอดนี้จะไม่ส่งบล็อกข้อมูลไปกับข้อความร้องขอเหมือนกับเมธอดเก็ท

เมธอดนี้มีประโยชน์สำหรับการตรวจสอบว่า มีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์หรือไม่ หรืออาจใช้เพื่อตรวจสอบความสมบูรณ์ของลิงก์ก็ได้ รหัสตอบสนองในบรรทัดสถานะจึงอาจเป็น 200 หรือ 400 และ อาจมีข้อมูลเพิ่มเติมมาในเซคเตอร์ด้วย เช่น วันที่ปรับปรุงแก้ไขไฟล์ครั้งสุดท้าย (Last Modified) เป็นต้น

สมมติว่ามีการส่งข้อความร้องขอโดยใช้เมธอดเฮดไปยังเซิร์ฟเวอร์ [ventura.lanna.com](http://ventura.lanna.com) เพื่อถามว่ามีไฟล์ `index.html` หรือไม่ โดยระบุข้อความร้องขอในบรรทัดแรกเป็น `HEAD/index.html` เอชทีทีพี (HTTP)/1.0 และสมมติว่ามีไฟล์ `index.html` อยู่ในเซิร์ฟเวอร์จริง และไฟล์นี้มีขนาด 1,938 ไบต์ ฉะนั้นข้อความตอบกลับจากเซิร์ฟเวอร์จะได้เป็นดังรูปที่ 2-6



รูปที่ 2-6 ข้อความตอบสนองที่ได้จากการร้องขอด้วยเมธอดเฮด



ในการร้องขอแบบเสตนั้น ข้อความตอบกลับจากการร้องขอจะไม่มีการส่งเนื้อหาของไฟล์มาให้ (ไม่มีบล็อกข้อมูล) ถึงแม้ว่าไฟล์ที่ร้องขอไปจะมีอยู่ในเซิร์ฟเวอร์จริงก็ตาม การสอบถามข้อมูลของไฟล์ในเซิร์ฟเวอร์ด้วยเมธอดเสด จึงทำงานได้เร็วกว่าใช้เมธอดเก็ต ตัวอย่างการใช้งานเมธอดเสดคือ มีบางโปรแกรมทำงานโดยให้ผู้ใช้กำหนดชื่อไฟล์ที่ต้องการจากเซิร์ฟเวอร์ หลังจากนั้นโปรแกรมจะคอยวิ่งไปเช็กับทางเซิร์ฟเวอร์เป็นช่วงๆ ตามระยะเวลาที่ตั้งไว้ (โปรแกรมร้องขอไปยังเซิร์ฟเวอร์ด้วยเมธอดเสด) และหากพบว่าไฟล์ในเซิร์ฟเวอร์มีการปรับปรุง คือ วันที่ของไฟล์ในเซิร์ฟเวอร์ใหม่กว่าไฟล์ที่เกี่ยวข้องในเครื่อง โปรแกรมจึงจะดาวน์โหลดไฟล์นั้นมาทับไฟล์เดิมในเครื่อง (ร้องขอไฟล์นั้นอีกครั้งด้วยเมธอดเก็ต) เป็นต้น

### 2.1.2.3.3 การร้องขอด้วยเมธอดโพส (POST)

การร้องขอด้วยเมธอดโพส (POST) จะใช้ในกรณีที่ต้องการส่งข้อมูลจากไคลเอนต์ไปยังเซิร์ฟเวอร์ คือ ไม่ได้ขอไฟล์จากเซิร์ฟเวอร์ แต่จะส่งข้อมูลไปให้เซิร์ฟเวอร์รับไปทำงานต่อแทน และเมื่อเซิร์ฟเวอร์รับข้อความร้องขอแบบโพส (POST) แล้วก็จะทราบว่ามีไคลเอนต์ต้องการส่งข้อมูลมาให้ รูปแบบข้อความร้องขอในบรรทัดแรกจะเขียนคล้ายกับเมธอดเก็ต ดังนี้

---

POST /path/file HTTP / x.x

---

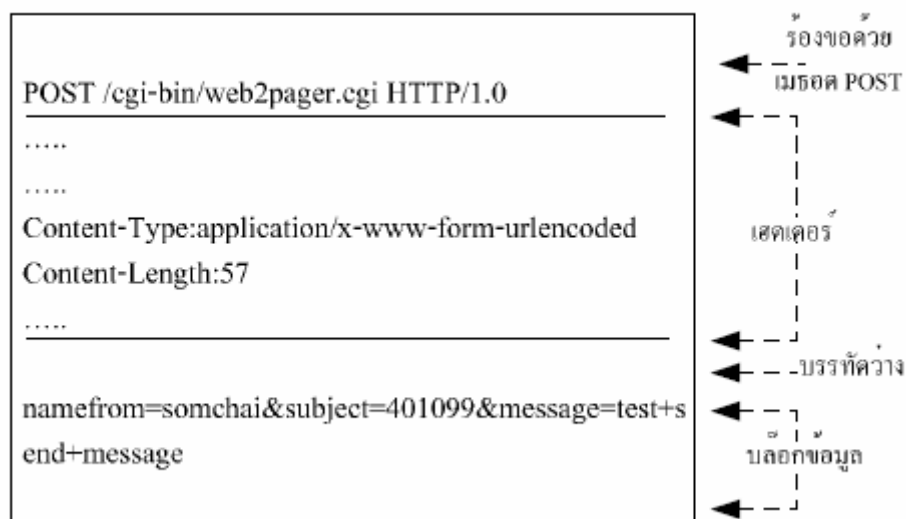
การร้องขอแบบโพสจะมีเงื่อนไขดังนี้

- ข้อมูลที่ส่งไปให้เซิร์ฟเวอร์จะอยู่ภายในบล็อกข้อมูลเท่านั้น ดังนั้นจึงต้องมีเฮดเดอร์เพื่อบอกรายละเอียดของข้อมูลในบล็อกแนบไปด้วย เช่น ข้อมูลเป็นแบบไหน (Content - Type) และข้อมูลมีกี่ไบต์ (Content - Byte) เสมอ
- /path/file คือชื่อโปรแกรม ซีจีไอ (CGI) ในเซิร์ฟเวอร์ที่จะทำหน้าที่รับข้อมูลไปประมวลผล
- ข้อความตอบสนองที่เซิร์ฟเวอร์จะส่งกลับมาให้ไคลเอนต์ จะได้มาจากการทำงานของโปรแกรมหรือ ซีจีไอ ในเซิร์ฟเวอร์ ดังนั้น ซีจีไอ ในเซิร์ฟเวอร์ที่รับข้อมูลไปจึงต้องทำหน้าที่ส่งข้อความกลับไปให้ไคลเอนต์

โดยส่วนใหญ่การส่งข้อมูลจากฟอร์มในเว็บเพจไปประมวลผลด้วยซีจีไอในเซิร์ฟเวอร์นั้นจะใช้เมธอดนี้มากที่สุด ความจริงแล้วการส่งข้อมูลไปยังเซิร์ฟเวอร์สามารถใช้เมธอดเก็ตก็ได้ แต่จะมีข้อจำกัดเรื่องความยาวของข้อมูลที่จะส่งไปให้เซิร์ฟเวอร์ โดยถ้าใช้เมธอดโพสเพื่อร้องขอข้อมูลส่งไปยังเซิร์ฟเวอร์ เฮดเดอร์ที่ชื่อ Content - Type จะถูกกำหนดให้เป็น application/x-www-form-urlencoded เพื่อบอกแก่เซิร์ฟเวอร์ว่าข้อมูลที่ส่งไปให้มีการเข้ารหัส ในส่วนเฮดเดอร์ Context - Length จะใช้สำหรับบอกความยาวของข้อมูลที่ทำกรเข้ารหัสแล้ว เพราะข้อมูลที่กรอกผ่านอินพุตในฟอร์มจะถูกเว็บเบราว์เซอร์เข้ารหัสก่อนส่งเสมอ การเข้ารหัสนี้เรียกว่า ยูอาร์แอลเอนโคด (URL - Encode) ซึ่งมีรายละเอียดดังนี้

- แปลงตัวอักษรหรือเครื่องหมายบางตัวให้อยู่ในรูป %xx โดยที่ xx จะเป็นค่ารหัสแอสกีของตัวอักษรตัวนั้นๆ ตัวอักษรที่ต้องมีการแปลง เช่น = , & , % และ + เพราะเป็นเครื่องหมายที่ใช้เป็นตัวแบ่งแยกข้อมูลที่จะส่งไปให้เซิร์ฟเวอร์
- เปลี่ยนช่องว่าง (Space) ทุกตัวเป็นเครื่องหมาย (+)
- รวมชื่อตัวแปรและค่าตัวแปรเข้าด้วยกัน โดยคั่นกลางด้วยเครื่องหมาย = และคั่นกลางระหว่างตัวแปรด้วยเครื่องหมาย & เช่น number=152-401099&from=zorkia&message=test

ตัวอย่างข้อความร้องขอด้วยเมธอดโพส ที่เว็บเบราว์เซอร์สร้างขึ้นเพื่อส่งข้อมูลจากฟอร์มเว็บเพจไปให้แก่ซีจีไอ ที่ชื่อ web2pager.cgi ในไดเรกทอรี /cgi/bin ของเซิร์ฟเวอร์ <http://161.246.5.35> รับประทานเป็นดังรูปที่ 2-7



รูปที่ 2-7 ตัวอย่างข้อความร้องขอด้วยเมธอดโพส

#### 2.1.2.4 ข้อความตอบสนอง (response)

ไม่ว่าจะมีข้อมูลหรือไม่ข้อมูลที่ไคลเอ็นต์ร้องขอเข้ามาก็ตาม เซิร์ฟเวอร์จะต้องส่งข้อความตอบสนองกลับไปให้ไคลเอ็นต์รับทราบเสมอ ในกรณีของการร้องขอข้อความบรรทัดแรกสุดจะบ่งบอกวิธีการร้องขอ แต่ถ้าเป็นกรณีตอบสนอง ข้อความบรรทัดนี้จะเรียกว่าเป็น “บรรทัดสถานะ” (status line) โครงสร้างในบรรทัดนี้มี 3 ส่วน แต่ละส่วนคั่นด้วยช่องว่าง ดังต่อไปนี้

---

HTTP /x.x xxx Description

---

- HTTP /x.x คือเวอร์ชันของเอชทีทีพี
- xxx ตัวเลข 3 หลักที่เป็นรหัสตอบสนอง (response status code)

- Description ข้อความอธิบายรหัสตอบสนอง ให้อ่านเข้าใจได้ง่ายขึ้น ตัวเลขหลักแรกของรหัสตอบสนอง เป็นรหัสบอกหมวดหมู่ของการตอบสนอง ดังต่อไปนี้
  - 1xx ข้อความข่าวสารอย่างเดียว
  - 2xx การร้องขอสำเร็จ
  - 3xx เปลี่ยนทิศทางการไหลของไคลเอนต์ไปยัง URL อื่น
  - 4xx มีความผิดพลาดที่เกิดจากการร้องขอ
  - 5xx มีความผิดพลาดทางด้านเซิร์ฟเวอร์รหัสตอบสนองที่ต้องเจอบ่อยๆ

ยกตัวอย่างเช่น

รหัส 200 หมายถึง ร้องขอสำเร็จ คือ มีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์ และเซิร์ฟเวอร์ได้ส่งเนื้อหาของไฟล์มาให้ไคลเอนต์ด้วยแล้ว

รหัส 400 หมายถึง ไม่มีไฟล์ที่ต้องการอยู่ในเซิร์ฟเวอร์

รหัส 500 หมายถึง มีความผิดพลาดในการทำงานของเซิร์ฟเวอร์  
เป็นต้น

ตัวอย่างเช่นถ้าเราป้อนURLให้กับเว็บเบราว์เซอร์เป็น

<http://www.ventura.lanna.com/news/current.html> ซึ่งเป็นการขอเปิดไฟล์ current.html ในไดเรกทอรี /news จากเซิร์ฟเวอร์ ventura.lanna.com และสมมุติว่าเซิร์ฟเวอร์มีไฟล์นี้ในเซิร์ฟเวอร์จริง ขนาดไฟล์เป็น 96 ไบต์ และเนื้อหาในไฟล์ current.html เป็นดังต่อไปนี้

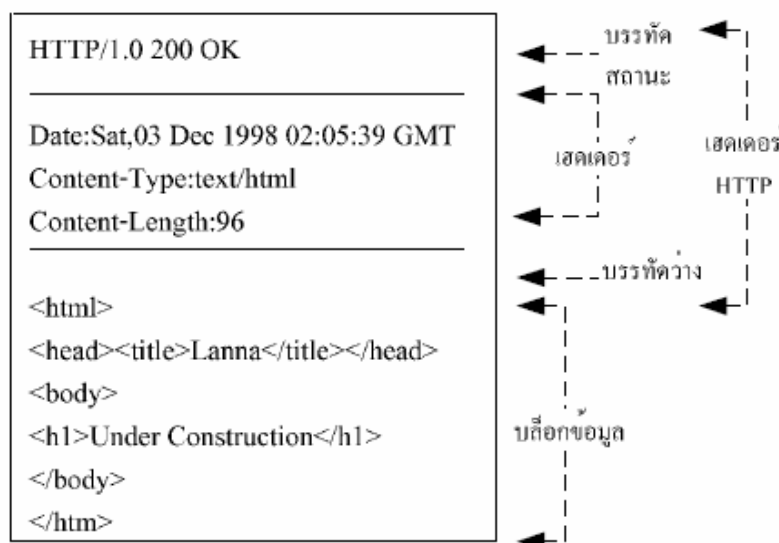
```
<html>
<head><title>Lanna</title></head>
<body>
<h1> Under Constuction </h1>
</body>
</html>
```

เมื่อเว็บเซิร์ฟเวอร์ได้รับคำร้องขอแล้ว ไปตรวจสอบพบว่ามีไฟล์ที่เว็บเบราว์เซอร์ต้องการอยู่จริง ดังนั้นเว็บเบราว์เซอร์จะส่งข้อความตอบสนองกลับ โดยที่ข้อความในบรรทัดสถานะจะเป็น

---

## HTTP/1.0 200 OK

---



รูปที่ 2-8 ข้อความตอบสนองที่ส่งมาจากเซิร์ฟเวอร์

และเนื้อหาของไฟล์ current.html นั้นจะถูกส่งมาให้เว็บเบราว์เซอร์ โดยจะอยู่ในบล็อกข้อมูล เพื่อให้เว็บเบราว์เซอร์นำไปตีความหมายและแสดงในวินโดวส์เว็บเบราว์เซอร์ ถ้าจำลองตัวเองเป็นเว็บเบราว์เซอร์หลังจากที่ส่งข้อความร้องขอไปยังเซิร์ฟเวอร์แล้ว อาจเห็นข้อมูลในลักษณะดังรูปที่ 2-8 ตอบกลับมาจากเซิร์ฟเวอร์

จากรูปที่ 2-8 จะเห็นได้ว่าในส่วนของเซคเตอร์ จะมีรายการเซคเตอร์อยู่ 3 บรรทัด บรรทัดแรกที่มีคำว่า Date : จะเป็นตัวบอกวัน เดือน ปี ที่ข้อความตอบสนองถูกสร้างขึ้น บรรทัดที่ 2 Context – type : บอกข้อมูลที่อยู่ที่ส่งมาอยู่ในบล็อกข้อมูลมีเนื้อหาแบบ text/html และบรรทัดสุดท้าย Context - Length : บอกจำนวนไบต์ของข้อมูลที่ส่งมา (จำนวนไบต์ของข้อมูลในบล็อกข้อมูล)

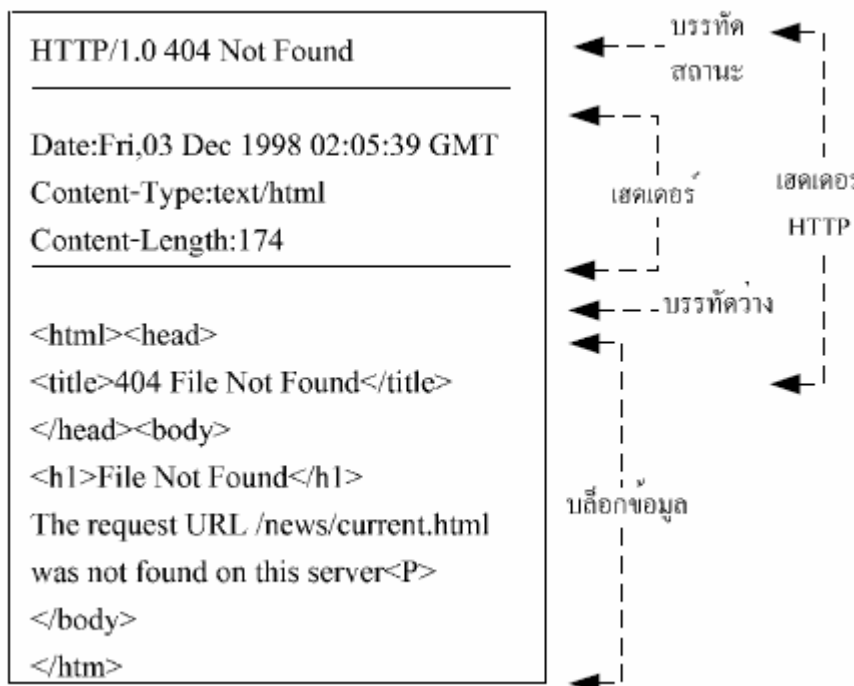
แต่หากว่าไม่มีไฟล์ current.html อยู่ที่เซิร์ฟเวอร์ ดังนั้นเซิร์ฟเวอร์ต้องตอบกลับให้เว็บเบราว์เซอร์ทราบว่าไม่มีไฟล์ดังกล่าว โดยข้อความในบรรทัดสถานะจากเหตุการณ์นี้จะเป็น

---

## HTTP /1.0 404 Not Found

---

ตัวเลขรหัส 404 คือ รหัสบอกว่ามีความผิดพลาดในการร้องขอ และคำว่าค้นหา Not Found เป็นคำอธิบายความหมายของรหัส 404 ฉะนั้นถ้าจำลองตัวเราเป็นเว็บเบราว์เซอร์อีกครั้ง ก็อาจเห็นข้อมูลลักษณะดังรูปที่ 2-8 ตอบกลับมาจากเซิร์ฟเวอร์



รูปที่ 2-9 ข้อความตอบสนองจากเซิร์ฟเวอร์เมื่อไม่มีข้อมูลร้องขอ

ในการตอบกลับการให้บริการของเซิร์ฟเวอร์ที่พื้นนั้นจะมีรูปแบบการใช้งานเหมือนโปรโตคอลอื่นๆ ตามหลักการของไคลเอนต์ - เซิร์ฟเวอร์ทั่วไป คือ จะมีการส่งค่ากลับมาด้วยหมายเลขที่เรียกว่า “Response Tag Number” หรือ “Status Code” และจะตามด้วยรายละเอียดข้อความซึ่งอธิบายหมายเลขนั้น จากนั้นส่วนท้ายสุดที่อาจจะมีส่งตามมาก็คือตัวของข้อมูลจริงๆ ในกรณีที่มีการขอข้อมูล เช่น จากการร้องขอข้างต้น เราจะได้ข้อมูลที่เว็บเซิร์ฟเวอร์ส่งกลับมาให้ดังนี้

```

Trying 127.0.0.0 ... Connected to localhost.
Escape character is '^]'
HTTP/1.1 200 OK
Date: Sun, 13 Aug 2000 17:17:34 GMT
Server: Apache/1.3.20
Content-type: text/html
Content-length: 1339
Last-modified: Mon, 05 Aug 2000 22:39:GMT
Connection: Keep_Alive
Keep-Alive: timeout=15, max=5
<HTML>
<HEAD><TITLE>KMITL Website</TITLE></HEAD>

```

&lt;BODY&gt;

:

&lt;/BODY&gt;

&lt;/HTML&gt;

หมายเลขการจะเป็นค่ามาตรฐานแต่ข้อความที่ตามมานั้นอาจจะไม่เหมือนกันก็ได้ทั้งนี้ขึ้นอยู่กับ  
ผลิตภัณฑ์ของเว็บเซิร์ฟเวอร์ ตัวเลขทุกหลักล้วนมีความหมาย

| Response Tags Number | รายละเอียด                    |
|----------------------|-------------------------------|
| 100                  | Continue                      |
| 101                  | Switching                     |
| 200                  | OK                            |
| 201                  | Created                       |
| 202                  | Accepted                      |
| 203                  | Non-Authoritative Information |
| 204                  | No Content                    |
| 205                  | Reset Content                 |
| 206                  | Partial Content               |
| 300                  | Multiple Choices              |
| 301                  | Moved Permanently             |
| 302                  | Moved Temporarily             |
| 303                  | See Other                     |
| 304                  | Not Modified                  |
| 305                  | Use Proxy                     |
| 400                  | Bad Request                   |
| 401                  | Unauthorized                  |
| 402                  | Payment Required              |
| 403                  | Forbidden                     |
| 404                  | Not Found                     |
| 405                  | Method Not Allowed            |
| 406                  | Not Acceptable                |
| 407                  | Proxy Authentication Required |
| 408                  | Request Time-out              |

|     |                            |
|-----|----------------------------|
| 409 | Conflict                   |
| 410 | Gone                       |
| 411 | Length Required            |
| 412 | Precondition Failed        |
| 413 | Request Entity Too Large   |
| 414 | Request-URI Too Large      |
| 415 | Unsupported Media Type     |
| 500 | Internal Server Error      |
| 501 | Not Implemented            |
| 502 | Bad Gateway                |
| 503 | Service Unavailable        |
| 504 | Gateway Time-out           |
| 505 | HTTP Version not supported |

ตาราง 2-1 รายละเอียดของหมายเลขสถานะการทำงานของโปรโตคอล HTTP

#### ข้อแตกต่างระหว่างเวอร์ชัน 1.0 กับเวอร์ชัน 1.1

โปรโตคอลเวอร์ชันใหม่ในเอชทีทีพีเวอร์ชัน 1.1 นี้ได้เพิ่มประสิทธิภาพทำงานให้สูงขึ้น และปรับปรุงในด้านต่างๆ ที่ทำให้มีความสามารถมากขึ้น ดังนี้

- ลดภาระของการเชื่อมต่อผ่านโปรโตคอลทีซีพี (TCP Protocol) และสามารถใช้อะประสิทธิภาพของทีซีพี (TCP) ได้เต็มที่
- สามารถทำการบีบอัดข้อมูลที่รับส่งระหว่างเซิร์ฟเวอร์และไคลเอนต์ได้
- รองรับการทำงานแบบ Virtual Host ซึ่งหมายถึง เว็บเซิร์ฟเวอร์เครื่องหนึ่งๆ มีชื่อโดเมนมากกว่า 1 ชื่อได้
- สามารถรองรับการทำงานได้หลายภาษา
- โอนไฟล์ข้อมูลเฉพาะบางส่วนได้ คุณสมบัตินี้มีประโยชน์มากในกรณีที่มีการโอนไฟล์ข้อมูลขนาดใหญ่ และเกิดปัญหาระหว่างการทำงาน ซึ่งโปรโตคอลเอชทีทีพีเวอร์ชัน 1.1 มีจุดเด่นที่สามารถตรวจสอบปัญหาได้ และสามารถโอนไฟล์ข้อมูลต่อจากส่วนที่เคยโอนมาแล้วได้

### 2.1.2.5 การเชื่อมต่อระหว่างผู้ให้บริการและผู้ให้บริการ

การเชื่อมต่อระหว่างผู้ให้บริการและผู้ให้บริการสามารถแบ่งออกได้เป็น 2 ลักษณะ คือ

#### 2.1.2.5.1 การเชื่อมต่อโดยตรง

ลักษณะการทำงานของการทำงานเชื่อมต่อโดยตรง คือ ผู้ขอใช้บริการจะติดต่อกับเซิร์ฟเวอร์หรือผู้ให้บริการโดยตรง ซึ่งส่วนของไคลเอนต์จะเรียกว่า User Agent โดยมีเว็บเบราว์เซอร์ทำหน้าที่นี้และส่วนเซิร์ฟเวอร์จะเรียกว่า Origin ซึ่งจะทำงานกับเว็บเบราว์เซอร์หรือ User Agent โดยตรง

#### 2.1.2.5.2 การเชื่อมต่อผ่านตัวกลาง

ลักษณะการติดต่อแบบนี้ส่วนของ User Agent ไม่สามารถติดต่อกับ Origin ได้โดยตรง นั่นคือต้องติดต่อผ่านตัวกลางทุกครั้งที่มีการร้องขอบริการและการตอบสนองก็ต้องผ่านตัวกลางเช่นกัน ดังนั้นการร้องขอหรือการตอบสนองจะมีลักษณะเหมือนลูกโซ่โยงผ่านเป็นช่วงๆ เรียกว่า Request Chain/Response Chain ประเภทของตัวกลาง (Intermediate) ตามข้อกำหนดของ HTTP มี 3 ประเภท คือ

- Tunnel: ทำหน้าที่เชื่อมต่อเท่านั้น อาจมีไว้เพื่อความประสงค์อะไรบางอย่างแต่ตัวกลางนี้จะไม่ทำหน้าที่ หรืออำนาจในการเปลี่ยนข้อมูลที่วิ่งผ่าน
- Proxy: ส่วนนี้สามารถปรับปรุงรายละเอียด มีการประยุกต์ใช้งานได้ทั้งสองส่วน คือ ไคลเอนต์และเซิร์ฟเวอร์ มักจะนำมาติดตั้งเป็น Cache Server หรือ FireWall
- Gateway: ส่วนนี้มักทำหน้าที่เชื่อมต่อ ในกรณีที่ไม่สามารถติดต่อ หรือใช้งานเชื่อมต่อกับตัวเซิร์ฟเวอร์ได้โดยตรง

### 2.1.3 ภาษาสคริปต์ PHP

ลักษณะของการเขียนเว็บเพจให้มีสคริปต์ PHP จะอาศัยวิธีการเขียนซอร์สโค้ดให้อยู่ในรูปแบบของภาษาสคริปต์ PHP ทั้งหมดเลยก็ได้ หรืออาจจะเขียนในรูปแบบการฝัง (Embed) คำสั่งหรือฟังก์ชันของ PHP ลงไปเฉพาะในตำแหน่งที่ต้องการ ซึ่งเหมือนกับการเขียนเว็บเพจทั่ว ๆ ไปที่มีการฝังสคริปต์ภาษา HTML นั่นเอง สคริปต์ PHP จะใช้แท็กในการกำหนดขอบเขตของสคริปต์ ซึ่งอาจเรียกว่า PHP Script Tag โดยประกอบด้วยแท็กเปิดและแท็กปิด แท็กเปิดของ PHP เขียนได้ 2 แบบ คือ <? หรือ <?php ส่วนแท็กปิดเขียนอยู่ในรูป ?> การนำวิธีการฝังสคริปต์มาใช้ ในการเขียนเว็บเพจกำลังเป็นที่นิยมอย่างมาก ทั้งนี้เพราะเป็นวิธีการเขียนเว็บเพจที่สะดวก ต่อผู้เขียนในการตรวจสอบการทำงานของเว็บเพจ โดยส่วนของเว็บเพจที่ไม่ได้กำกับด้วยสคริปต์ใด ๆ ก็จะแสดงผลไปตามข้อความนั้น ๆ โดยตรง หากเราจะเปลี่ยนแปลงแก้ไขข้อความใด ๆ ก็สามารถกระทำได้เลย โดยไม่ต้องกังวลว่าเว็บเพจจะทำงานไม่ถูกต้อง และเมื่อเว็บเพจแจ้งข้อความว่าเกิดข้อผิดพลาด อันเนื่องมาจากการทำงานของสคริปต์ เราก็เพียงไปแก้ไขหรือปรับปรุงเฉพาะจุดที่เป็นสคริปต์นั้น ๆ เช่น ตัวอย่าง

```
<html>
  <head>
    <title>Example</title>
  </head>
```



```

<body>

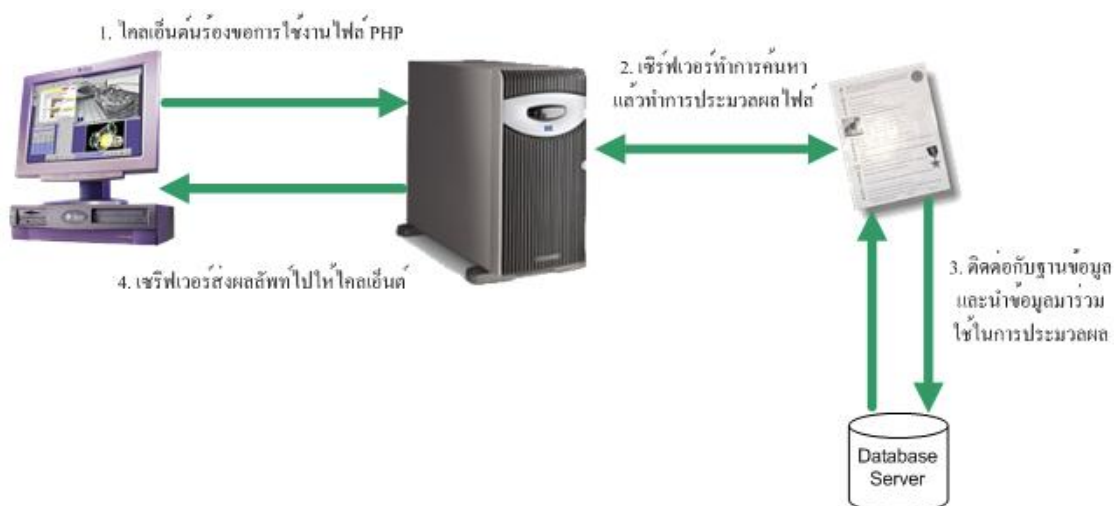
<?php
    echo "Hi, I'm a PHP script!";
?>

</body>
</html>

```

### 2.1.3.1 โครงสร้างสถาปัตยกรรม

ข้อแตกต่างของสคริปต์ PHP กับสคริปต์ภาษา HTML คือ สคริปต์ PHP เป็นเซิร์ฟเวอร์ไซด์สคริปต์ ( Server Site Script ) โดยถูกเรียกให้ทำงานฝั่งเซิร์ฟเวอร์ ส่วนสคริปต์ภาษา HTML เป็นไคลเอนต์ไซด์สคริปต์ ( Client Side Script ) นั่นคือ สคริปต์ จะถูกเรียกทำงานทางฝั่งไคลเอนต์ หรือฝั่งของเบราว์เซอร์ สคริปต์ PHP มีการทำงานดังรูป



รูปที่ 2-10 การทำงานของสคริปต์ PHP

1. ฝั่งไคลเอนต์จะทำการร้องขอหรือเรียกใช้งานไฟล์ PHP ที่เก็บไว้ในเครื่องเซิร์ฟเวอร์
2. ฝั่งเซิร์ฟเวอร์จะทำการค้นหาไฟล์ PHP แล้วทำการประมวลผลไฟล์ PHP ตามที่ฝั่งไคลเอนต์ทำการร้องขอมา
3. ทำการประมวลผลไฟล์ PHP
4. ถ้ามีการติดต่อกับฐานข้อมูล ฝั่งเซิร์ฟเวอร์ก็จะทำการติดต่อกับฐานข้อมูลโดยส่งคำสั่งเอสคิวแอล (SQL) ไปยังฐานข้อมูลและนำข้อมูลในฐานข้อมูลมาใช้ในการประมวลผลนั้นด้วย
5. ประมวลผลคำสั่ง (SQL) แล้วส่งค่าของข้อมูลที่ต้องการกลับไปยังเครื่องเซิร์ฟเวอร์
6. ส่งผลลัพธ์จากการประมวลผลไปให้เครื่องไคลเอนต์

## คุณสมบัติเด่นที่ทำให้เลือกใช้ PHP

- PHP มีการ Compile และ Execute ได้อย่างรวดเร็ว
- ซอร์สโค้ดของ PHP เป็นแบบโอเพ่นซอร์ส (Open source) โดยเปิดโอกาสให้โปรแกรมเมอร์ทั่วไปเข้ามามีส่วนร่วมในการพัฒนา ทำให้มีการพัฒนาได้เร็วขึ้นและมีคนใช้งานจำนวนมาก ข้อดีของ Open source ก็คือจะมีการแก้ไขข้อผิดพลาดและเพิ่มคุณสมบัติเด่นๆ ให้ตรงตามความต้องการของตลาดอยู่ตลอดเวลา PHP สามารถดาวน์โหลดได้ฟรีจากเว็บไซต์ <http://www.php.net>
- เนื่องจาก PHP ทำงานบนเครื่องเว็บเซิร์ฟเวอร์ ดังนั้น โปรแกรมที่เขียนขึ้นโดย PHP สามารถที่จะมีขนาดใหญ่และมีความซับซ้อนได้สูงโดยไม่กระทบต่อการทำงานของเครื่อง Client เลย
- ไม่ขึ้นกับระบบใดๆ (Cross-platform) โดย PHP สามารถรองรับการทำงานได้บน Web Server ทั้งในระบบ Windows, Unix, Macintosh ไม่ว่าจะเป็น Apache, IIS, Netscape Enterprise, OmniHttpd และอื่นๆ PHP สามารถทำงานได้ในเกือบจะทุก ๆ ระบบปฏิบัติการ เช่น Unix, Linux และ Windows
- เนื่องจากโค้ดของ PHP เป็นแบบเอ็มเบ็ดเด็ด (Embedded) ซึ่งสามารถที่จะฝังไว้ในโค้ด HTML เลย จึงเป็นการง่ายต่อการเรียนรู้ใช้งาน
- Protocol Support โดย PHP นั้นสามารถเขียนเพื่อเชื่อมต่อและทำงานกับโปรโตคอลได้หลากหลาย ไม่ว่าจะเป็น IMAP, SNMP, NNTP, POP3, HTTP, Ftp, Socket ฯลฯ
- Open API โดยผู้ใช้สามารถเขียนโปรแกรมที่เป็นโมดูลเพิ่มเติม ในการทำงานที่เฉพาะออกไปอีกได้
- PHP ยึดติดอยู่กับหลักการพื้นฐาน โครงสร้างของภาษาไม่ซับซ้อนเหมือนกับภาษา C หรือ ภาษา JAVA ที่มีความซับซ้อนกว่า แต่ถึงกระนั้นตัว PHP เอง ก็มีความสามารถเพียงพอที่จะสนับสนุนการทำงานของเว็บไซต์ทุกๆ ขนาด
- PHP ใช้ทรัพยากรของระบบน้อยมากเมื่อเทียบกับภาษาอื่น
- PHP สามารถรองรับระบบฐานข้อมูลที่หลากหลาย ที่มีใช้กันอยู่ในปัจจุบัน ไม่ว่าจะเป็น Empress, DB2, Informix, InterBase, mSQL, MySQL, MS-SQL, ODBC, Oracle, Sybase, PostgreSQL และระบบฐานข้อมูลอื่น ๆ ที่สนับสนุนมาตรฐาน ODBC โดยไม่จำเป็นต้องอาศัยเครื่องมืออื่นช่วย ในขณะที่หากใช้ภาษาอื่นเช่น ASP หรือ Visual Basic นั้น การเข้าถึงข้อมูลใน MySQL นั้นจะต้องผ่าน ODBC
- PHP มีฟังก์ชันที่ใช้จัดการเกี่ยวกับระบบไฟล์มากมาย
- PHP สามารถใช้งานทางด้านกราฟฟิคได้ ไม่ว่าจะเป็นการสร้างรูปเหลี่ยมต่างๆ กราฟแท่ง กราฟวงกลม และอื่นๆ อีกมาก
- PHP มีฟังก์ชันมากมายที่จะจัดการกับข้อความ รวมถึงความสามารถในการเปรียบเทียบรูปแบบของข้อความ

- PHP สนับสนุนตัวแปรแบบต่าง ๆ มากมาย ทั้งสเกลาร์และอาร์เรย์
- PHP ได้รับการสนับสนุนจากผู้ใช้อย่างแพร่หลายทั่วโลก ทำให้ PHP มีการพัฒนาอย่างต่อเนื่อง และมีผู้ใช้อยู่มากมายทั่วโลก ทำให้เมื่อพบปัญหาต่าง ๆ เราจะสามารถหาข้อมูลได้อย่างสะดวก

### 2.1.3.2 Predefined variable

Predefined variable เป็นตัวแปรที่มีการกำหนดไว้ล่วงหน้าเรียบร้อยแล้ว ซึ่งผู้ใช้สามารถทำการเรียกใช้ได้ในทันทีภายในสคริปต์ที่กำลังใช้งานอยู่ ซึ่งตัวแปรเหล่านี้จะขึ้นอยู่กับเว็บเซิร์ฟเวอร์ที่ใช้งานอยู่ด้วย เพราะถ้าต่างชนิดกัน ตัวแปรก็จะแตกต่างกันออกไปแต่โดยภาพรวมแล้วก็จะมียกข้อยกเว้นที่คล้ายๆ กัน ซึ่งในตัวอย่างที่ศึกษานี้จะเป็น Predefine variable ที่ใช้กับ Apache 1.3.6 โดยแสดงให้เห็นเพียงบางส่วนเท่านั้น ซึ่งในกรณีที่ต้องการดู Predefine variable ทั้งหมดสามารถดูได้โดยการเรียกใช้ฟังก์ชันดังนี้

```
<?php
phpinfo()
?>
```

ซึ่งฟังก์ชัน phpinfo() นี้จะแสดง Predefined variable ทั้งหมดพร้อมค่าต่างๆ ที่ได้ทำการเซตไว้ ซึ่งจะมีประโยชน์ในการติดตามและปรับแต่งค่าให้เหมาะสมยิ่งขึ้น แต่ควรใช้ด้วยความระมัดระวังเพราะถ้าเกิดผู้บุกรุกได้เข้ามาเห็นการปรับแต่งค่าในไฟล์นี้ อาจจะเป็นแนวทางในการหาวิธีเจาะเข้ามายังในระบบได้ก็เป็นได้

Predefined variables นี้มีความสำคัญมากในการนำไปใช้งาน ซึ่งถ้าไม่มีการปรับแต่งที่เหมาะสม และการเรียกใช้งานอย่างถูกต้องแล้ว อาจจะเป็นช่องโหว่ที่ผู้บุกรุกจะเข้ามาภายในระบบได้ ซึ่งจะกล่าวรายละเอียดเกี่ยวกับเรื่องการปรับแต่งและการนำไปใช้งานต่อไปในภายหลัง โดย Predefined variables นี้จะแบ่งออกเป็น 2 ประเภทหลักๆ ด้วยกันคือ

#### 1. Apache variables

ซึ่งตัวแปรเหล่านี้จะสร้างขึ้นมาโดยอพาเซในขณะที่ยกใช้สคริปต์ ซึ่งตัวแปรที่สำคัญมีดังนี้

##### **\$SERVER\_NAME**

เป็นชื่อของเซิร์ฟเวอร์ที่สคริปต์ที่เราเรียกใช้กำลังใช้งานอยู่ เช่น "ISAG16"

##### **\$SERVER\_PROTOCOL**

เป็นข้อมูลเกี่ยวกับ protocol ของหน้าที่เว็บไซต์ทำการร้องขอ เช่น "HTTP/1.1"

##### **\$REQUEST\_METHOD**

จะแสดงข้อมูลว่าวิธีการใดที่ใช้ในการร้องขอเพื่อดูเว็บไซต์นั้น เช่น "GET", "HEAD", "POST", "PUT"

##### **\$DOCUMENT\_ROOT**

จะแสดง root directory ที่สคริปต์นั้นกำลังทำงานอยู่

**\$HTTP\_CONNECTION**

จะแสดง Connection header ของหน้าเพจปัจจุบันที่เรียกดู เช่น “keep Alive”

**\$REMOTE\_ADDR**

แสดง IP address ของผู้ใช้ที่เรียกดูข้อมูลจากเว็บไซต์

**\$REMOTE\_PORT**

แสดงถึง port ที่เครื่องของผู้ใช้ในการติดต่อกับ web server

**\$SERVER\_PORT**

แสดงถึง port บนเครื่อง server โดยปกติจะใช้ port 80 ซึ่งอาจจะมีการเปลี่ยนไปใช้ port อื่นได้ เช่น ในกรณีที่ทำการติดตั้ง secure HTTP เพื่อใช้ SSL อาจเปลี่ยนไปใช้ port อื่นได้ตามความเหมาะสม

**2. Environment variable**

ตัวแปรเหล่านี้จะสร้างขึ้นมาโดย PHP เองซึ่งจะถือเป็นตัวแปร โกลบอลสามารถเรียกใช้ได้ทุกที่ในขณะที่เรียกใช้สคริปต์ ซึ่งตัวแปรที่สำคัญมีดังนี้

**\$argv**

เป็นอะเรย์ของ argument ต่างๆ ที่ทำการส่งมาให้กับ PHP สคริปต์

**\$PHP\_SELF**

เป็นชื่อไฟล์ของสคริปต์ที่กำลังเรียกใช้อยู่ในปัจจุบัน

**\$HTTP\_COOKIE\_VARS**

เป็นอะเรย์ของตัวแปรที่ส่งมาให้สคริปต์ผ่านทาง HTTP cookies

**\$HTTP\_GET\_VARS**

เป็นอะเรย์ของตัวแปรที่ส่งมาให้สคริปต์โดยวิธี GET ผ่านทาง HTTP

**\$HTTP\_POST\_VARS**

เป็นอะเรย์ของตัวแปรที่ส่งมาให้สคริปต์โดยวิธี POST ผ่านทาง HTTP

**\$HTTP\_ENV\_VARS**

เป็นอะเรย์ของตัวแปรที่ส่งมาให้สคริปต์ผ่านทาง parent environment

**\$HTTP\_SERVER\_VARS**

เป็นอะเรย์ของตัวแปรที่ส่งมาให้สคริปต์โดย HTTP server

**\$HTTP\_POST\_FILES**

เป็นอะเรย์ของตัวแปรเกี่ยวกับการ upload file โดยวิธี POST ผ่านทาง HTTP ที่ส่งมาให้สคริปต์

สำหรับ \$HTTP\_\*\_VARS จะสามารถใช้งานได้ก็ต่อเมื่อมีการเซตให้ “track\_var = on” ในไฟล์ php.ini ซึ่งเมื่อมีการเซตแล้วก็จะมีการเกิดขึ้นแม้ว่าจะไม่มีการส่งตัวแปรมาให้สคริปต์ก็ตาม ก็จะมีการ

กำหนดเป็น empty อะเรย์ไว้ เพื่อป้องกันผู้ที่ไม่หวังดีทำการปลอมตัวแปรและค่าของตัวแปรได้ ซึ่ง track\_vars นี้จะมีค่า on เสมอใน PHP 4.03 เป็นต้นไป แม้ว่าจะมีการ set ค่าไปเป็นอย่างอื่นก็ตาม

ในไฟล์ php.ini ถ้ามีการเซตค่า “register\_global=on” ตัวแปรเหล่านี้ทุกตัวจะมีการกำหนดและสามารถเรียกใช้ได้เสมือนเป็นตัวแปร global ซึ่งแตกต่างจาก HTTP\_\*\_VARS โดยสามารถเรียกใช้ได้โดยเสมือนตัวแปรทั่วไป เช่น \$name, \$username ซึ่งค่า register\_global นี้ควรใช้ด้วยความระมัดระวัง ถ้าเป็นไปได้ควรเซตให้เป็น off ซึ่งตัวแปร \$HTTP\_\*\_VARS จัดได้ว่าเป็นตัวแปรที่มีความปลอดภัยเพราะเป็นตัวแปรที่มีค่าจริงๆ ในขณะที่ตัวแปร global ธรรมดาสามารถเปลี่ยนค่าใหม่โดยผู้ใช้ได้ซึ่งนับเป็นจุดที่ควรระมัดระวังเป็นอย่างมาก ซึ่งถ้ามีสิ่งให้ “register\_global=on” จะต้องมีการตรวจสอบอินพุตให้แน่ใจก่อนว่าเป็นค่าที่รับมาอย่างถูกต้องจริงๆ ก่อนจะนำไปใช้งานต่อไป

### 2.1.3.3 Regular Expression

Regular Expression หรือ Pattern Matching เป็นการประมวลผลและจัดการรูปแบบเฉพาะของข้อมูลในข้อความต่างๆ ซึ่งจะมีประโยชน์ดังต่อไปนี้

1. ใช้สำหรับตรวจสอบข้อมูลบางอย่างที่ต้องการ เพื่อให้ตรงกับความต้องการของข้อมูลที่จะนำไปใช้ เช่น การตรวจสอบอีเมลล์ของผู้ใช้ที่พิมพ์เข้ามา ว่าต้องอยู่ในรูปแบบของอีเมลล์จริงๆ การตรวจสอบรูปแบบตัวอักษร ตัวเลข หรือสัญลักษณ์ต่างๆ

2. ใช้สำหรับค้นหาหรือแทนที่ข้อมูลที่มีปริมาณมากๆ เช่น ตัวแปรที่เก็บข้อความไว้จำนวนมาก หรือข้อความในไฟล์ทั้งไฟล์

ฟังก์ชันที่ใช้ในการเปรียบเทียบที่สำคัญคือ ereg ซึ่งมีรูปแบบการใช้งาน ดังนี้

**Ereg** (รูปแบบที่ต้องการค้นหา,รูปแบบที่มีอยู่)

โดยฟังก์ชัน ereg จะคืนค่า true ถ้าตรวจพบรูปแบบที่ต้องการค้นหา และคืนค่า false ถ้าไม่พบรูปแบบที่ต้องการค้นหา โดยในการนำไปใช้งานนั้น จะนำไปใช้ร่วมกับ if เพื่อทำการเปรียบเทียบ เช่น

```
<?php
    $str = "Computer Engineering";
    if (ereg("Computer",$str)) {
        print "Yes found";
    } else {
        print "Not found";
    }
?>
```

จากตัวอย่างโปรแกรมนั้น ได้ทำการค้นหาคำว่า Computer ในตัวแปร \$str ว่ามีอยู่หรือเปล่า ซึ่งผลการค้นหาพบว่ามีอยู่ ผลลัพธ์จึงพิมพ์คำว่า Yes found ออกมา

### พื้นฐาน Pattern Matching

Pattern จะหมายถึงคำหรือข้อความที่เราต้องการที่จะค้นหา ซึ่งจากตัวอย่างข้างบน Pattern จะเป็นคำว่า Computer แต่ในการใช้งานจริงๆ แล้ว เราสามารถกำหนดรูปแบบของ Pattern ได้ว่าจะให้รูปแบบของการค้นหานั้นออกมาเป็นอย่างไร โดยจะมีการนำเครื่องหมายเข้ามาช่วย เพื่อให้ Pattern ในการค้นหามีความยืดหยุ่นมากขึ้น

ในการตรวจสอบตัวอักษรจำนวน 1 ตัวว่ามีตัวอักษรนี้อยู่หรือเปล่านั้นจะใช้รูปแบบ [...] เข้ามาช่วย เช่น

|             |                                                                     |
|-------------|---------------------------------------------------------------------|
| [a]         | ตรวจสอบว่ามีตัวอักษร a อยู่หรือไม่                                  |
| [R]         | ตรวจสอบว่ามีตัวอักษร R อยู่หรือไม่                                  |
| [a-z]       | ตรวจสอบว่ามีตัวอักษร a-z ตัวใดตัวหนึ่งอยู่หรือไม่                   |
| [A-Z]       | ตรวจสอบว่ามีตัวอักษร A-Z ตัวใดตัวหนึ่งอยู่หรือไม่                   |
| [a-zA-Z0-9] | ตรวจสอบว่ามีตัวอักษร a-z หรือ A-Z หรือ 0-9 ตัวใดตัวหนึ่งอยู่หรือไม่ |
| [+ - * /]   | ตรวจสอบว่ามีเครื่องหมาย +,-,*,/ ตัวใดตัวหนึ่งอยู่หรือไม่            |

การตรวจสอบจุดเริ่มต้นและจุดสิ้นสุด จะใช้รูปแบบ ^...\$ โดยเครื่องหมาย ^ จะใช้ในการตรวจสอบจุดเริ่มต้นและ \$ ใช้ในการตรวจสอบจุดสิ้นสุด ดังตัวอย่างต่อไปนี้

|                        |                                                              |
|------------------------|--------------------------------------------------------------|
| ^computer              | ตรวจสอบว่าข้อความนั้นขึ้นต้นด้วยคำว่า computer หรือเปล่า     |
| network\$              | ตรวจสอบว่าข้อความนั้นขึ้นลงท้ายด้วยคำว่า network หรือเปล่า   |
| ^network programming\$ | ตรวจสอบว่าข้อความนั้นเป็นคำว่า network programming หรือเปล่า |

การตรวจสอบตัวอักษรที่ซ้ำกัน จะใช้เครื่องหมาย {X} โดย X คือจำนวนของตัวอักษรที่ซ้ำกันได้ ตัวอย่าง เช่น

|         |                                                                  |
|---------|------------------------------------------------------------------|
| ^g{3}\$ | ตรวจสอบว่าข้อความนั้นต้องเป็น ggg                                |
| g{3}    | ตรวจสอบว่าข้อความนั้นต้องประกอบด้วย ggg โดยอยู่ที่ตำแหน่งใดก็ได้ |
| t{2,4}  | ตรวจสอบว่าข้อความนั้นมี tt, ttt, tttt ตัวใดตัวหนึ่งอยู่หรือไม่   |

การใช้เครื่องหมาย . และเครื่องหมาย + โดยเครื่องหมาย . จะหมายถึงตัวอักษรใดๆ ก็ได้ ส่วนเครื่องหมาย + จะหมายถึงว่าสามารถมีตัวอักษรนั้นได้มากกว่า 1 ตัว ตัวอย่าง เช่น

|          |                                                     |
|----------|-----------------------------------------------------|
| ^. {2}\$ | ตรวจสอบว่ามีตัวอักษร 2 ตัวใดๆ ก็ได้ เช่น ab, rr, ty |
| ^+.\$    | ตรวจสอบข้อความอะไรก็ได้ (ยกเว้นการขึ้นบรรทัดใหม่)   |

ตัวอย่างการนำความรู้ทางด้าน Regular Expression ไปประยุกต์ใช้ เช่น การตรวจสอบอีเมลที่รับเข้ามาอย่างง่าย โดยทั่วไปอีเมลจะอยู่ในรูป [XXX@XXX.XXX](#) ซึ่งสามารถเขียน Regular Expression ตรวจสอบได้ดังนี้คือ `^.+@.+\.+$` โดยมีตัวอย่างโปรแกรมดังนี้

```
<?php
    if (ereg("^.+@.+\.+$", $email)) {
        print "Email is correct";
    } else {
        print "Email not correct";
    }
?>
```

### 2.1.4 หลักการทำงานของ CGI

CGI ก็คือหลักการหรือวิธีการของการพัฒนาแอปพลิเคชัน ที่ทำหน้าที่เสมือนประตู (Gateway) เชื่อมโยงการติดต่อกับการทำงานอื่นๆ เพื่อให้เกิดการทำงานที่หลากหลายในการใช้งาน โดยอาศัยพื้นฐานของระบบเว็บหรือจะกล่าวได้ว่าทำงานควบคู่กับเว็บเซิร์ฟเวอร์ เพราะบราวเซอร์ไม่สามารถติดต่อส่วนอื่นๆ โดยตรงได้ เช่น จะติดต่อกับฐานข้อมูล เป็นต้น จำเป็นต้องติดต่อผ่านเว็บเซิร์ฟเวอร์ ไปยังส่วนของ CGI โดยเรามักเรียกว่า “CGI โปรแกรม” หรือ “CGI แอปพลิเคชัน” หรือ “เว็บแอปพลิเคชัน” ก็ได้ ด้วยเหตุนี้เองเราจึงเห็นว่าจริงๆ แล้ว CGI แอปพลิเคชัน หรือ แอปพลิเคชันที่พัฒนาตามแนวทาง CGI เป็นแอปพลิเคชันประเภทเซิร์ฟเวอร์แอปพลิเคชัน (Server Application) หรือแอปพลิเคชันที่ทำงานอยู่ที่ฝั่งเซิร์ฟเวอร์ โดยมีส่วนที่ทำหน้าที่ติดต่อกับผู้ขอบริการหรือไคลเอนต์ คือเว็บเซิร์ฟเวอร์ และไคลเอนต์ใช้งานผ่านเว็บเบราว์เซอร์ ข้อดีของเซิร์ฟเวอร์แอปพลิเคชันก็คือ การปรับปรุงหรือเปลี่ยนเวอร์ชันจะทำได้ง่าย โดยไม่ต้องแจกจ่ายให้ผู้ใช้งานทุกครั้งแต่สามารถดูแลปรับปรุงได้ที่เซิร์ฟเวอร์โดยตรง พอมีวิธีการของ CGI เกิดขึ้นปัจจุบันเราจึงได้เห็นรูปแบบของโฮมเพจที่เปลี่ยนไปจากเดิมที่เคยเป็นแค่เอกสารที่แสดงโดยไม่มีการเปลี่ยนแปลง (Static Hypermedia Document) ไปเป็นเอกสารที่สามารถเปลี่ยนแปลงรูปแบบได้ ตลอดจนเห็นเป็นโฮมเพจที่สามารถโต้ตอบหรือเป็นอินเตอร์แอคทีฟ (Interactive) เหมือนส่วนของอินเตอร์เฟซ (Interface) ของ CGI แอปพลิเคชัน ที่แปรเปลี่ยนตลอดเหมือนกับการใช้งานโปรแกรมประยุกต์นั่นเอง

ลักษณะการทำงานของซีจีไอ (CGI) ต้องอาศัยการประมวลผลที่เซิร์ฟเวอร์ แล้วสร้างคำตอบออกมาเป็นเนื้อหาแบบเอชทีเอ็มแอล จากนั้นจึงส่งเนื้อหากลับไปให้ไคลเอนต์ ซึ่งเป็นการทำงานแบบรวมศูนย์ คือ งานทุกอย่างทำที่เซิร์ฟเวอร์หมด ไคลเอนต์เพียงแค่ทำหน้าที่ส่งคำร้องขอ และรอรับผลการทำงานเท่านั้น ดังนั้นเซิร์ฟเวอร์จะต้องทำงานหนักมาก และใช้เนื้อที่ในการเก็บข้อมูลเยอะ แต่เพื่อเป็นการลดภาระให้แก่เซิร์ฟเวอร์ เราสามารถใช้ Cookie ช่วยได้โดยให้ทางฝั่งไคลเอนต์ช่วยเก็บข้อมูลของผู้ใช้แต่ละคนไว้แทน

### 2.1.5 หลักการ Web Database

ในการเขียนโปรแกรมบนเว็บในปัจจุบัน ส่วนมากจะต้องมีการเก็บข้อมูลบางอย่างเอาไว้ เพื่อนำไปใช้ต่อไป ซึ่งการเขียนระบบฐานข้อมูลด้วยตัวเองนั้นจะต้องออกแบบรูปแบบของการเก็บข้อมูลเอง และในการนำข้อมูลจากฐานข้อมูลไปใช้นั้นย่อมเกิดความผิดพลาดได้ถ้าการเขียนโปรแกรมไม่รัดกุมพอ

ในการเขียนโปรแกรมบนเว็บในยุคแรก ๆ การเก็บข้อมูลนั้น โดยมากจะใช้เท็กซ์ไฟล์ในการเก็บ จะพบได้ว่าการเขียนโปรแกรมควบคุมเท็กซ์ไฟล์ เช่น โปรแกรมเอดิเตอร์ หรือ โปรแกรมประมวลผลข้อมูล ในไฟล์ที่ใช้เท็กซ์ไฟล์ เป็นฐานข้อมูลนั้น การควบคุมเท็กซ์ไฟล์นั้นลำบากกว่าการควบคุมไบนารีที่มีฟิลด์ และ เรคคอร์ดเข้ามาช่วยควบคุม และโอกาสในการเกิดข้อผิดพลาดในการควบคุมเท็กซ์ไฟล์นั้นมากกว่า เมื่อเว็บไซต์ เป็นแหล่งรวบรวมข้อมูลที่มีคุณค่า การใช้ระบบจัดการฐานข้อมูลเข้ามาช่วยจัดการกับข้อมูลต่าง ๆ จึงเป็นอีกทางเลือกหนึ่งที่ทำให้การบริหารข้อมูลบนเว็บไซต์มีความสะดวกมากขึ้น และโอกาสผิดพลาดก็จะมีน้อยลง โปรแกรมที่จะทำหน้าที่เป็นตัวกลางในการเรียกใช้ข้อมูลจากฐานข้อมูล และนำข้อมูลมาแสดงบนเว็บไซต์นั้น ก็คือโปรแกรมที่สร้างจากสคริปต์ PHP ดังที่ได้กล่าวไว้ข้างต้นนั่นเอง

#### คุณสมบัติเด่นที่ทำให้เลือกใช้ MySQL

- MySQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ซึ่งจะทำการเก็บข้อมูลแยกเป็นตารางแทนที่จะเก็บข้อมูลไว้รวม ๆ กันเป็นก้อนใหญ่ไว้ในที่หนึ่ง ซึ่งสิ่งนี้ได้เพิ่มความเร็วและความยืดหยุ่นในการใช้งานฐานข้อมูล ตารางเหล่านี้จะเชื่อมกันโดยการกำหนดความสัมพันธ์ให้แก่แต่ละตาราง ซึ่งจะทำให้สามารถรวมข้อมูลจากหลาย ๆ ตารางได้
- MySQL ใช้ภาษา SQL ( Structured Query Language ) เป็นพื้นฐานในการกระทำการต่าง ๆ กับฐานข้อมูล ซึ่งภาษา SQL นี้เป็นภาษามาตรฐานในการติดต่อกับฐานข้อมูลอยู่แล้ว ทำให้ผู้ใช้สามารถเรียนรู้การใช้งาน MySQL ได้อย่างง่ายดายและรวดเร็ว
- MySQL เป็นซอร์สแบบเปิด กล่าวคือ ใคร ๆ ก็ตามต่างก็มีสิทธิ์ใช้ MySQL ได้โดยไม่ต้องเสียค่าใช้จ่ายใด ๆ ทั้งสิ้น อีกทั้งผู้ใช้สามารถเรียนรู้การทำงานของ MySQL ได้จากซอร์สโค้ด และสามารถทำการแก้ไขซอร์สโค้ดนั้นเพื่อให้ MySQL มีความเหมาะสมกับความต้องการของตนได้
- MySQL มีความสามารถในการสืบค้นข้อมูลได้อย่างถูกต้องและรวดเร็ว และมีประสิทธิภาพ
- สามารถใช้ MySQL ได้ในหลาย ๆ ระบบปฏิบัติการ เช่น Linux, Unix, Windows
- MySQL ง่ายต่อการเรียนรู้และใช้งาน

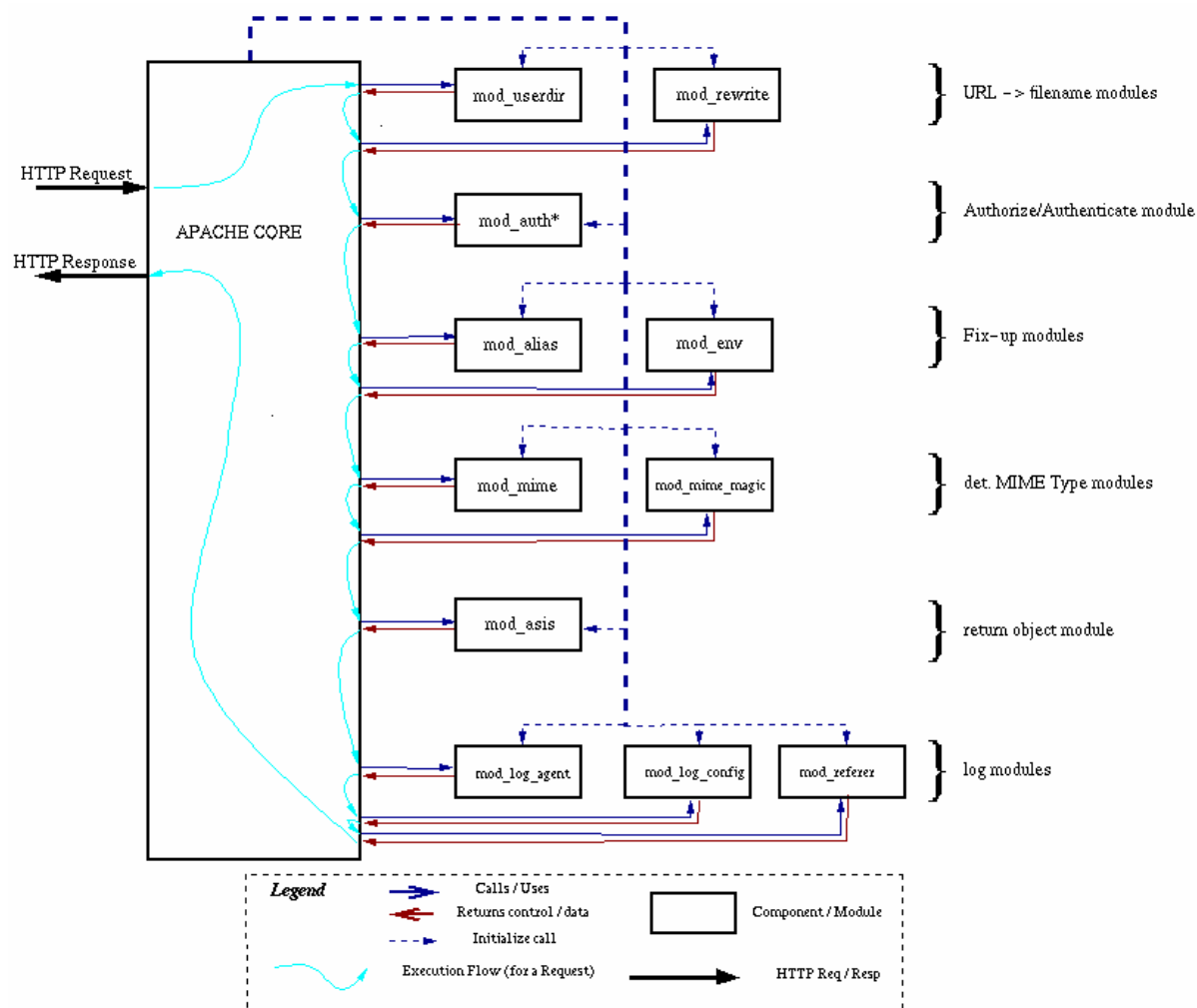
### 2.1.6 เว็บเซิร์ฟเวอร์ อปาเช่

อปาเช่ (Apache) คือ โปรแกรมสร้างระบบเว็บเซิร์ฟเวอร์ หรือ HTTP เซิร์ฟเวอร์ที่สามารถทำงานได้ทั้งบนระบบลินุกซ์ ระบบยูนิกซ์ รวมทั้งระบบวินโดวส์อื่น ๆ ได้ด้วย อปาเช่นั้นมีต้นกำเนิดมาจากโปรแกรม NCSA httpd 1.3 และได้รับการพัฒนาและปรับปรุงเรื่อยมา จนถึงได้ว่าเป็นโปรแกรมระบบเว็บ



เซิร์ฟเวอร์ที่ดีตัวหนึ่ง ในปัจจุบัน โดยมีจุดเด่นทั้งในด้านความเร็ว มีความเชื่อถือได้ของโปรแกรมสูงมาก และมีความสามารถต่าง ๆ อย่างหลากหลายที่โปรแกรมอื่นเองต้องนำเอาไปเป็นแบบอย่าง

อาปาเช่ เป็นโปรแกรมเว็บเซิร์ฟเวอร์ที่ได้รับความนิยมสูงสุดเป็นอันดับต้นๆ ของโลก ในบรรดาโปรแกรมระบบเซิร์ฟเวอร์ของเว็บทั้งหลาย ซึ่งมีการสำรวจและจัดเก็บสถิติแล้วพบว่า กว่า 50% ของเครื่องคอมพิวเตอร์ต่าง ๆ ที่ทำงานเป็นระบบเว็บเซิร์ฟเวอร์นั้น ทำงานด้วยโปรแกรม อาปาเช่



รูปที่ 2-11 สถาปัตยกรรมโครงสร้างของ Apache อย่างละเอียด

คุณสมบัติเด่นที่ทำให้เลือกใช้อาปาเช่

- เป็นโปรแกรมระบบเว็บเซิร์ฟเวอร์ตามมาตรฐานโปรโตคอล HTTP/1.1
- มีระบบโมดูลให้ผู้ใช้สามารถเขียนโปรแกรมเพิ่มเติมความสามารถให้กับอาปาเช่ได้เอง ซึ่งในปัจจุบันมีผู้ใช้ทั้งหลายได้เขียนโมดูลต่าง ๆ ออกมาเป็น Third – party module อย่างมาก

- สนับสนุน SSL (Secure Sockets Layer) คือ Apache SSL ที่เป็นแบบไม่เป็นทางการค้า (Noncommercial) และในเวอร์ชันที่เป็นแบบทางการค้า (Commercial)
- มีขั้นตอนการเซตไฟล์ต่างๆ ในระบบง่าย และสามารถปรับแต่งการทำงานได้หลากหลาย
- สนับสนุนการตรวจสอบโปรโตคอล HTTP ผ่านทาง Text File, DBM File และทาง SQL หรือจากโปรแกรมภายนอก (External Program)
- มีระบบ DBM หรือ databases for authentication ให้เรากำหนดรหัสผ่านสำหรับอนุญาต และป้องกันการเรียกดูเพจต่าง ๆ ของโฮมเพจแต่ละหน้าให้เฉพาะผู้ใช้ที่ต้องการและไม่ต้องการได้
- มีความสามารถในการติดตามผู้ใช้จากคุกกี้ (Cookie)
- สนับสนุนการตรวจสอบในรูปแบบของ Message Digest
- สนับสนุนการทำงานแบบ Virtual Host ทั้งแบบ IP-Based หรือ Name-Based
- สามารถสั่งให้ส่งไฟล์หรือรัน CGI Script เมื่อเกิดข้อขัดข้องต่าง ๆ ได้ด้วย
- มีระบบ Multiple Directory Index คือ สามารถกำหนดชื่อไฟล์เพื่อเชื่อมกับ URL ที่เป็นแบบไคเรกทอรีได้มากกว่าหนึ่งชื่อไฟล์
- มีระบบ Content negotiation คือ อาปาเช่ สามารถปรับระดับความซับซ้อนของข้อมูลในเอกสาร HTML ที่จะส่งออกไป ให้สอดคล้องกับความสามารถของโปรแกรมเว็บเบราว์เซอร์ที่ทำงานบนเครื่อง Client ที่ติดต่อมาได้โดยอัตโนมัติ
- มีระบบ Multiple-homed servers ความสามารถนี้เป็นที่ต้องการอย่างยิ่งในปัจจุบัน คือ อาปาเช่ สามารถตอบสนองต่อเครื่อง Client ต่าง ๆ ให้ดูเหมือนเป็นเว็บเซิร์ฟเวอร์หลาย ๆ เครื่องพร้อมกันได้โดยติดตั้ง อาปาเช่ ให้กับเครื่องเซิร์ฟเวอร์เพียงเครื่องเดียว

#### ข้อเสียและลักษณะด้อยของ Apache

- ขาดรูปแบบการเซตแบบ GUI
- ไม่มีการสนับสนุนจากผู้ขายในกรณีที่เป็นแบบ Free Ware
- เกิดความไม่แน่นอนเมื่อมีการทำงานบนวินโดวส์แพลตฟอร์ม เนื่องจาก Apache มีการใช้ระบบ Multi-Threading อาจทำให้ทั้งระบบเกิดหยุดทำงานได้

## 2.2 ทฤษฎีที่ใช้ในการออกแบบฐานข้อมูล

ฐานข้อมูล (Database) คือ ที่เก็บข้อมูลและความสัมพันธ์ระหว่างข้อมูลเหล่านั้น โดยมีรูปแบบในการนำเสนอข้อมูลและความสัมพันธ์ระหว่างข้อมูล (Data model) หลายอย่างซึ่งระบบการจัดการข้อมูล (DBMS) ใช้การนำเสนอข้อมูลในรูปแบบของรีเลชันนัล (relational data model) คือ ใช้ตารางในการนำเสนอข้อมูลและความสัมพันธ์ระหว่างข้อมูล

## 2.2.1 ลักษณะที่ดีของระบบฐานข้อมูล

### 2.2.1.1 ความซ้ำซ้อนของข้อมูลน้อยที่สุด (Minimum Redundancy)

ความซ้ำซ้อน คือ ความสัมพันธ์ระหว่างข้อมูลที่เป็นจริง (Fact) ปรากฏมากกว่า 1 ครั้งในฐานข้อมูล โดยแบ่งออกเป็น 3 ชนิดคือ

- ความซ้ำซ้อนบนแถวเดียวกัน (Intra Row Redundancy)
- ความซ้ำซ้อนในตารางเดียวกัน (Intra Table Redundancy)
- ความซ้ำซ้อนในหลายตาราง (Inter Table Redundancy)

ข้อดีในการลดความซ้ำซ้อนของข้อมูล

- ใช้สโตร์เรจ(Storage) ได้อย่างมีประสิทธิภาพคือไม่เปลืองเนื้อที่ในการจัดเก็บข้อมูลที่ซ้ำซ้อน
- ไม่ต้องแก้ไขเปลี่ยนแปลงข้อมูลหลายที่ ทำให้สะดวกและข้อมูลไม่เกิดความขัดแย้ง (ลดปัญหา Multiple Update)

ข้อเสียในการลดความซ้ำซ้อนของข้อมูล

- อาจทำการตอบคำถามได้ช้าลงเพราะอาจต้องมีการ JOIN ตาราง (join table)

### 2.2.1.2 ความถูกต้องสูงที่สุด (Maximum Integrity)

ความถูกต้องถูกกำหนดโดยกฎด้านความถูกต้อง (Integrity Rule) ซึ่งมี 2 แบบคือ

- สถิตติก (Static) คือการบังคับความถูกต้องของสถานะของข้อมูล
- ไดนามิก (Dynamic) คือการบังคับความถูกต้องของการเปลี่ยนสถานะของข้อมูล

### 2.2.1.3 โปรแกรมควรเป็นอิสระจากเปลี่ยนแปลงโครงสร้างข้อมูล

โครงสร้างข้อมูลแบ่งเป็น 3 ระดับคือ

- อินเทอร์เน็ต สกีม (Internal Schema) เป็นระดับของการจัดเก็บข้อมูลจริงๆ (Physical level)
- คอนเซ็ปชวล สกีม (Conceptual Schema) เป็นระดับที่มองเอนติตี้ (entity) และความสัมพันธ์ระหว่าง เอนติตี้ทั้งหมด รวมทั้งกฎเกณฑ์ต่างๆที่เกี่ยวข้องกับข้อมูล และสิทธิในการใช้งาน (Logical level)
- เอ็กซ์เทอร์นอล สกีม (External Schema) เป็นระดับของข้อมูลที่จะมองเห็นจากการใช้งานของผู้ใช้แต่ละคน

ความเป็นอิสระของข้อมูล (Data Independence) มี 2 ลักษณะคือ

- ลอจิคอล (Logical Data Independence) คือ การแก้ไขเปลี่ยนแปลงข้อมูลระดับลอจิคอลไม่มีผลกระทบต่อโครงสร้างข้อมูลในระดับของเอ็กซ์เทอร์นอล ผลที่ได้คือไม่ต้องแก้ไขโปรแกรม

- ฟิสิกคอลล (Physical Data Independence) คือ การแก้ไขเปลี่ยนแปลงข้อมูลในระดับฟิสิกคอลล ไม่มีผลกระทบต่อโครงสร้างข้อมูลในระดับลอจิคอลล และเอ็กซ์เทอร์นอลล ผลที่ได้คือ ไม่ต้องแก้ไขโปรแกรม

#### 2.2.1.4 ความปลอดภัยสูง (High degree of data security)

สามารถเพิ่มความปลอดภัยในการใช้และเข้าถึงข้อมูลได้ เช่น

- การกำหนดชื่อผู้ใช้และรหัสผ่าน
- การสร้างวิว (Views) สำหรับการเข้าถึงข้อมูล
- การกำหนดสิทธิในการเข้าถึงข้อมูล

#### 2.2.1.5 การควบคุมจากศูนย์กลางแบบลอจิคอลล (Logically Centralized Control)

มีทีมงานที่ทำหน้าที่ในการควบคุมดูแลการใช้ข้อมูลจากศูนย์กลาง

### 2.2.2 ลักษณะของฐานข้อมูลแบบรีเลชันนัล

มีส่วนประกอบ 3 อย่างด้วยกันคือ

#### 2.2.2.1 โครงสร้างข้อมูล

โครงสร้างข้อมูลอยู่ในรูปของตารางเท่านั้น (Table only หรือ Relation only) โดยตารางต้องมีคุณสมบัติคือ

- ในแถวไม่มีชื่อคอลัมน์ที่ซ้ำกัน
- ข้อมูลที่เก็บในแถวเดียวกันไม่มีลำดับ คือ สามารถสลับคอลัมน์ก่อนหลังได้
- ค่าใน 1 คอลัมน์ต้องเป็นค่าที่ไม่สามารถแยกย่อยต่อไปอีกได้

#### 2.2.2.2 กฎความถูกต้อง (Integrity Rule)

ชนิดต่างๆของคีย์

- แคนดิเดทคีย์ คือ กลุ่มของแอททริบิวต์ที่มีค่าไม่ซ้ำกัน
- คีย์หลัก (Primary Key) คือ แคนดิเดทคีย์ตัวหนึ่งที่ถูกเลือกขึ้นมาโดยเอนติตีจะมีความถูกต้องเมื่อคีย์หลักไม่เป็น null (Entity Integrity)
- อัลเทอร์เนตคีย์ คือ แคนดิเดทคีย์ที่ไม่ใช่คีย์หลัก
- คอมบายนคีย์คือคีย์ที่ประกอบด้วยแอททริบิวต์มากกว่า 1 แอททริบิวต์ขึ้นไป

- คีย์รอง (Foreign Key) คือแอททริบิวต์ที่ไม่ใช่คีย์ในรีเลชันหนึ่งแต่เป็นคีย์หลักในอีกรีเลชันหนึ่งหรือรีเลชันเดียวกัน โดยจะมีความถูกต้องเมื่อค่าคีย์รองเหมือนกับค่าของคีย์หลักหรือเป็น Null (Referential Integrity)

### 2.2.2.3 ภาษาที่ใช้ในการจัดการข้อมูล (Data Manipulation Language)

ภาษาที่ใช้ในการจัดการข้อมูลต้องเป็น Relational Complete Language คือต้องมีความสามารถอย่างน้อยเทียบเท่ากับภาษา Relational Algebra หรือ ภาษา Relational Calculus

## 2.3 การออกแบบฐานข้อมูลโดยใช้ E-R Diagram

ในการออกแบบฐานข้อมูลขึ้นมาใช้งานในระบบสารสนเทศใดๆจะต้องอาศัยแบบจำลองของข้อมูล เพื่อนำเสนอรายละเอียดต่างๆ ที่เกี่ยวข้องกับข้อมูลในฐานข้อมูลที่ออกแบบ เนื่องจากแบบจำลองของข้อมูลจะมีรูปแบบในการนำเสนอรายละเอียดต่างๆที่เกี่ยวข้องกับฐานข้อมูลที่เป็นมาตรฐาน จึงทำให้สามารถนำเสนอต่อผู้ใช้ในแต่ละระดับที่มีมุมมองที่แตกต่างกันได้เป็นอย่างดี สำหรับแบบจำลองของข้อมูลที่นิยมใช้ได้แก่

### 2.3.1 เอนติตี (Entity)

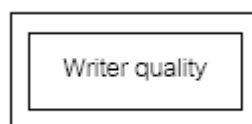
เป็นรูปภาพที่ใช้แทนคลาส ที่สามารถระบุได้ในความเป็นจริง ซึ่งอาจเป็นสิ่งที่จับต้องได้ เช่น พนักงาน หรืออาจเป็นเพียงสิ่งที่อยู่ในรูปนามธรรมที่ไม่สามารถจับต้องได้ เช่น วันหยุดของพนักงาน แบ่งออกเป็น 2 ประเภทดังนี้

1. Regular Entity ได้แก่เอนติตีที่ประกอบด้วยสมาชิกที่มีคุณสมบัติซึ่งบ่งบอกถึงเอกลักษณ์ของแต่ละสมาชิกรวมทั้งสำหรับรูปภาพที่ใช้แทนเอนติตีประเภทนี้ได้แก่รูปสี่เหลี่ยมผืนผ้า โดยมีชื่อของเอนติตตินั้นอยู่ภายใน

ARTICLE

รูปที่ 2-12 สัญลักษณ์ที่ใช้แทน Regular Entity

2. Weak Entity เป็นเอนติตีที่มีลักษณะตรงกันข้ามกับ Regular Entity กล่าวคือ สมาชิกของเอนติตีประเภทนี้จะสามารถมีคุณสมบัติที่บ่งบอกถึงเอกลักษณ์ของแต่ละสมาชิกได้นั้น จะต้องอาศัยคุณสมบัติใดคุณสมบัติหนึ่งของ Regular Entity มาประกอบกับคุณสมบัติของตัวเอง สำหรับรูปภาพที่ใช้แทนเอนติตีประเภทนี้ได้แก่รูปสี่เหลี่ยมผืนผ้า 2 รูปซ้อนกัน โดยมีชื่อของเอนติตตินั้นอยู่ภายใน

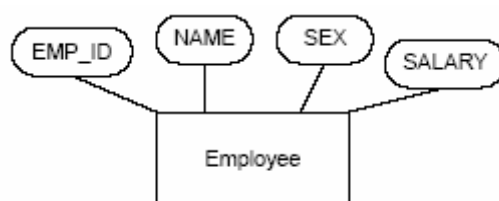


รูปที่ 2-13 สัญลักษณ์ที่ใช้แทน *Weak Entity*

### 2.3.2 แอตทริบิวต์ (Attribute)

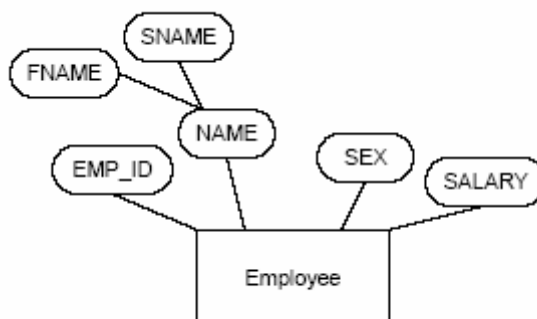
ได้แก่ คลาสของคุณสมบัติต่างๆ ที่นำมารวมกับแบบ Aggregation Abstraction เพื่อเป็นเอนติตี หรือ Relationship เช่น หมายเลขบัตรประชาชน ชื่อสกุล วันเดือนปีเกิด ภูมิลำเนา วันที่ออกบัตร วันที่บัตรหมดอายุ ที่รวมกันเป็นเอนติตี “บัตรประชาชน” เป็นต้น สำหรับ แอตทริบิวต์ ใน E-R Model สามารถแบ่งออกได้ 6 ประเภทดังนี้

1. Simple Attribute ได้แก่ แอตทริบิวต์ ที่ค่าภายใน แอตทริบิวต์ นั้นไม่สามารถแบ่งย่อยได้อีก เช่น เพศ เงินเดือน อายุ จังหวัด ฯลฯ เป็นต้นสำหรับรูปที่ใช้แทน แอตทริบิวต์ ประเภทนี้ ได้แก่ วงรีที่มีเส้นเชื่อมต่อไปยัง เอนติตี ที่เป็นเจ้าของ แอตทริบิวต์ นั้น โดยมีชื่อของ แอตทริบิวต์ นั้นอยู่ภายใน



รูปที่ 2-14 สัญลักษณ์ที่ใช้แทน *Simple Attribute*

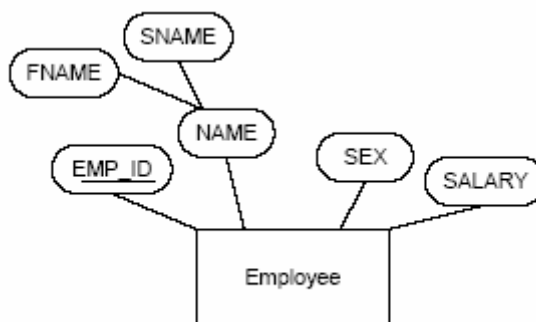
2. Composite Attribute เป็น แอตทริบิวต์ ที่มีลักษณะตรงข้ามกับ Simple Attribute กล่าวคือ จะเป็นแอตทริบิวต์ ที่ค่าภายใน แอตทริบิวต์ นั้นสามารถแยกเป็น แอตทริบิวต์ ย่อยได้อีก เช่น ชื่อ ที่สามารถแบ่งย่อยออกเป็น “คำนำหน้าชื่อ” “ชื่อ” “นามสกุล” สำหรับรูปภาพที่ใช้แทนแอตทริบิวต์ประเภทนี้ จะใช้วงรีเช่นเดียวกับ Simple Attribute แต่จะเป็นวงรีที่เชื่อมต่อกับวงรีของ Simple Attribute ที่เป็นเจ้าของ



รูปที่ 2-15 สัญลักษณ์ที่ใช้แทน *Composite Attribute*

### 2.3.3 Identifier หรือ Key

ได้แก่แอตทริบิวต์หรือกลุ่มของแอตทริบิวต์ที่มีค่าในแต่ละสมาชิกของเอนติตีไม่ซ้ำกันเลย ซึ่งถูกนำมาใช้กำหนดความเป็นเอกลักษณ์ให้กับแต่ละสมาชิกในเอนติตี สำหรับรูปภาพที่ใช้แทนคีย์ของเอนติตี จะใช้รูป วงรีเช่นเดียวกับแอตทริบิวต์แต่จะมีเส้นขีดอยู่ใต้แอตทริบิวต์ที่เป็นคีย์



รูปที่ 2-16 สัญลักษณ์ที่ใช้แทน Identifier หรือ Key

### 2.3.4 Single valued Attribute

เป็นแอตทริบิวต์ที่มีค่าของข้อมูลภายใต้ค่าของแอตทริบิวต์ใดแอตทริบิวต์หนึ่งเพียงค่าเดียว เช่น แอตทริบิวต์ “SALARY” ซึ่งที่ใช้เก็บเงินเดือนของพนักงาน โดยพนักงานแต่ละคนจะมีเงินเดือนค่าเดียว สำหรับรูปภาพที่ใช้แทนแอตทริบิวต์ ประเภทนี้จะใช้รูปภาพเช่นเดียวกับ Simple Attribute

### 2.3.5 Multi valued Attribute

เป็นแอตทริบิวต์ที่มีลักษณะตรงข้ามกับแอตทริบิวต์แบบ Single valued กล่าวคือ เป็นแอตทริบิวต์ที่มีค่าของข้อมูลได้หลายค่าภายใต้ค่าของแอตทริบิวต์ใดแอตทริบิวต์หนึ่ง เช่น แอตทริบิวต์ “DEGREE” ที่ใช้ระบุระดับการศึกษาของพนักงานแต่ละคน ซึ่งพนักงานแต่ละคนจะมีระดับการศึกษาได้หลายระดับ สำหรับรูปภาพที่ใช้แทนแอตทริบิวต์ประเภทนี้จะใช้รูปภาพเดียวกับ Simple Attribute แต่เส้นที่ใช้เชื่อมระหว่างรูปภาพของแอตทริบิวต์ประเภทนี้กับรูปภาพของเอนติตี หรือ Relationship จะใช้เส้น 2 เส้นแทน

### 2.3.6 Derived Attribute

เป็นแอตทริบิวต์ที่ค่าของข้อมูลได้มาจากการนำเอาค่าของแอตทริบิวต์อื่น มาทำการคำนวณ ซึ่งค่าของแอตทริบิวต์ประเภทนี้จะต้องเปลี่ยนแปลงทุกครั้งเมื่อมีการเปลี่ยนแปลงค่าของแอตทริบิวต์ที่ถูกนำมาคำนวณ เช่น แอตทริบิวต์ “TOT\_SAL” ของเอนติตี “EMPLOYEE” ที่ใช้เก็บเงินเดือนสะสมของพนักงานแต่ละคนเพื่อนำไปคำนวณภาษี ซึ่งได้มาจากผลรวมของค่าในแอตทริบิวต์ “INCOME” ของเอนติตี “MTHLY\_SALARY” ซึ่งเป็นเงินเดือนที่พนักงานแต่ละคนได้รับในแต่ละเดือน สำหรับรูปภาพที่ใช้แทนแอตทริบิวต์ประเภทนี้จะใช้รูปภาพเดียวกับ Simple Attribute แต่เส้นเชื่อมระหว่างรูปภาพของแอตทริบิวต์ประเภทนี้กับรูปภาพของเอนติตี หรือ Relationship จะใช้เส้นปะแทน

### 2.3.7 Relationship

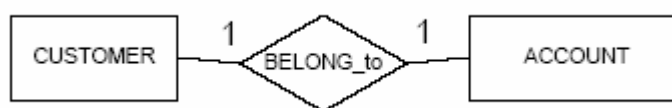
ได้แก่ การนำเอาเอนติติมารวมกันแบบ Aggregation Abstraction ดังนั้น สมาชิกของ Relationship จึงเกิดจากการจับคู่กันระหว่างสมาชิกของเอนติติที่มารวมกันภายใต้ Relationship นั้น Relationship ที่สร้างขึ้นนั้นจะใช้แทนความสัมพันธ์ใดความสัมพันธ์หนึ่งระหว่างสมาชิกของเอนติติที่มารวมกันภายใต้ Relationship นั้น Relationship ระหว่างเอนติติใดๆ ไม่จำเป็นที่จะต้องมียัง Relationship เดียว ถ้าความสัมพันธ์ระหว่างสมาชิกในเอนติติเหล่านั้น มีมากกว่า 1 ความสัมพันธ์



รูปที่ 2-17 Relationship

#### 2.3.7.1 One to One Relationship

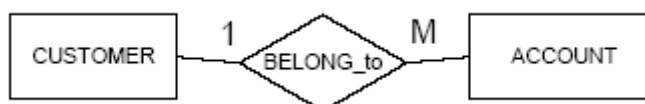
เป็น Relationship ที่แต่ละ Participant ของเอนติติหนึ่ง จะมีความสัมพันธ์กับอีก Participant ของอีกเอนติติหนึ่งเพียง participant เดียว



รูปที่ 2-18 ความสัมพันธ์แบบ One to One

#### 2.3.7.2 One to Many Relationship

เป็น Relationship ที่แต่ละ Participant ของเอนติติหนึ่งมีความสัมพันธ์กับ Participant ของอีกเอนติติหนึ่งมากกว่า 1 Participant เช่น กรณีลูกค้าสามารถมีบัญชีเงินฝากได้มากกว่า 1 บัญชี และแต่ละบัญชีเงินฝากจะต้องมีเจ้าของบัญชีเพียงคนเดียว



รูปที่ 2-19 ความสัมพันธ์แบบ One to Many



### 2.3.7.3 Many to Many Relationship

เป็น Relationship ที่ participant มากกว่า 1 participant ของเอนติตีหนึ่ง มีความสัมพันธ์กับ participant ของอีกเอนติตีหนึ่งมากกว่า 1 participant เช่น กรณีลูกค้าสามารถมีบัญชีเงินฝากได้มากกว่า 1 บัญชี และแต่ละบัญชีเงินฝากสามารถมีเจ้าของบัญชีได้มากกว่า 1 คน



รูปที่ 2-20 ความสัมพันธ์แบบ Many to Many

นอกเหนือจากการใช้จำนวนของ Participant ในการจัดประเภทของ Relationship แล้ว ยังสามารถใช้จำนวนของเอนติตีที่มีความสัมพันธ์กับแต่ละ Relationship มากำหนดประเภทของ Relationship ได้ดังนี้

1. Binary Relationship เป็น Relationship ที่พบมากที่สุด ใน E-R Diagram โดยเป็น Relationship ที่เกิดขึ้นระหว่าง 2 เอนติตี ใดๆ เช่น E-R Diagram ที่แสดงความสัมพันธ์ระหว่างลูกค้ากับบัญชีเงินฝาก เป็นต้น
2. N-ary Relationship เป็น Relationship ที่เกิดขึ้นระหว่างเอนติตี มากกว่า 2 เอนติตี ขึ้นไป เช่น Relationship “SCHEDULE” ซึ่งใช้แสดงตารางเรียนของวิชาต่างๆ
3. Recursive Relationship เป็น Relationship ที่เกิดขึ้นกับ เอนติตี เดียว ในกรณีที่ แอททริบิวต์ของเอนติตตินั้น สามารถสร้างความสัมพันธ์กับอีกแอตทริบิวต์หนึ่งภายในเอนติตีเดียวกัน

### 2.3.8 การแปลง E-R Diagram เป็น Relational Table

1. แปลง regular entity type เป็น 1 ตาราง โดยแอตทริบิวต์ที่ แปลง ไปจะเป็น แอททริบิวต์ทุกชนิดยกเว้น multivalued attribute
2. แปลง weak entity type เป็น 1 ตาราง ในลักษณะเดียวกับข้อ 1 คีย์หลักจะเป็น patian key combine กับ คีย์หลักของ parent
3. 1:1 relationship เป็น primary key foreign key โดยไม่ต้องสร้างตารางใหม่ ถ้าฝั่งใดเป็น total ให้ใช้ฝั่ง total เป็นหลัก ยก คีย์หลักของอีกฝั่งหนึ่งมาต่อเป็น foreign key พร้อมด้วยแอตทริบิวต์ของความสัมพันธ์นั้น
4. 1:M relationship ไม่ต้องสร้างตารางใหม่ ใช้ primary key foreign key โดยใช้ฝั่ง many เป็นหลัก ยก คีย์หลัก ฝั่ง 1 มาต่อเป็น foreign key พร้อมด้วยแอตทริบิวต์ของความสัมพันธ์
5. M:M relationship เป็น 1 ตาราง โดยคีย์หลักเป็น combine key ของ primary key ที่เกี่ยวข้อง